

دلوقتي بناء علي القتاير المرفوعه ليك وكمان المحادثات الخاصه بالمشروع عايز بنك تقرير احترافي

يوضح عملنا ايه وايه اللي لسه متعملشي وايه اللي اتعمل ومش مظلوط عايزك منك تترتب اللي

مفروض يتعمل للمشروع بالترتيب كعقليه مهندس ميكارونكس

ممتاز

بشكل احترافي فعلًا **odometry** دلوقتي إحنا دخلنا في مرحلة تحليل أداء الـ

نمن كلامك

بحرك العربية بعدها بدقيقتين لما بيان التأثير

هل الصح إن الفريمين يبقوا في نفس المستوى؟

والاستجابة بطيئة جدًا

خلينا نفصلهم هندسيًا واحدة واحدة

يبقوا في نفس المستوى؟ **base_link** و **odom** أولًا: هل طبيعي إن

الإجابة المختصرة: آه، ده الصح

ليه؟

نعلى أرض مستوية differential drive في روبوت

● **odom** → frame مرجعي ثابت (world-like frame)

● **base_link** → مركز الروبوت

● $Z = 0$ لاتنين المفروض يكونوا في مستوى

يعني

odom (0,0,0)

↓

base_link (x,y,0)

uneven terrain. مافيش ارتفاع بينهم إلا لو عندك روبوت طائر أو

الوضع عندك سليم، TF فمن ناحية الـ

ثانيًا: البطء الشديد (دقيقتين علشان يظهر التأثير)

ده مش طبيعي إطلاقًا

على الأقل Hz المفروض تتحدث بسرعة 10-50 odometry الـ

لو التأثير بيظهر بعد دقيقتين، يبقى عندنا مشكلة في أحد 3 أماكن

تشخيص المشكلة كخبير روبوتات

odom /الخطوة 1 — نقيس تردد

جديد واكتب Terminal افتح

ros2 topic hz /odom

وحرك العربة

النتائج المتوقعة

التفسير	النتيجة
ممتاز	أو 20 Hz 30 Hz
مقبول	10 Hz
فيه مشكلة	أقل من 1 Hz
كارثة	0.01 Hz

قولي الرقم اللي بيطلع

الخطوة 2 — راقب سلوك الرسائل

افتح:

ros2 topic echo /odom

وحرك الروبوت

سأل نفسك:

- هل القيم ستتغير كل شوية صغيرة؟
- ولا بتفضل صفر وبعدين فجأة تقفز؟

ESP32 لو بتقفز → يبقى البيانات جاية متأخر من

RViz بطيء → يبقى إعدادات RViz لو بتتغير تدريجي لكن

أهم نقطة غالبًا هي السبب

ESP32 راجع سكيتش الـ

غالبًا عندك حاجة زي

```
if (millis() - lastTime >= 500) {  
    lastTime = millis();  
    Serial.print("L=");  
    Serial.print(leftPos);  
    Serial.print(" R=");  
    Serial.println(rightPos);  
}
```

ms لو الرقم 500

يبقى الإرسال كل نص ثانية → ده بطيء جدًا

ms لو 1000

يبقى مرة كل ثانية → أسوأ

5000 ms لو

يبقى كارثة

المعدل الصحيح

الـ odometry لازم تبعت كل

10ms 50 ms إلى

يعني

```
if (millis() - lastTime >= 20) {
```

20ms = 50Hz

وده ممتاز

ليه البطء يحصل رياضياً؟

الـ encoder_odometry node يتحسب

$\Delta ticks = new_ticks - last_ticks$

لو الرسالة جاية كل ثانية

● لدلتا بتبقى كبيرة

● لتحديث يحصل مرة كل ثانية

● RViz يظهر حركة فجائية

● delay تحس إن فيه

delay... لكن الحقيقة مش

البطيء update rate هي

بعد ما تزيط معدل الإرسال

ارجع اعمل

ros2 topic hz /odom

لأزم تشوف حاجة زي

average rate: 25.0

متبقى لحظة RViz ساعتها الحركة في

نقطة احترافية إضافية

نفسه لازم يكون عندك encoder_odom_node في الـ

```
timer_ = this->create_wall_timer(
```

```
std::chrono::milliseconds(20),
```

```
std::bind(&EncoderOdomNode::update, this));
```

1000 ms أو 500 ms لوقتته

يبقى لازم يتعدل

الخلاصة

حالته الجزء

سليم المستوى TF

سليم frame relationship

سببها معدل تحديث بطيء الاستجابة البطيئة

الحل	قلل زمن الإرسال في ESP32 timer أو في node
------	---

لوقتتي عايز منك

1. نتيجة ros2 topic hz /odom

2. كام؟ ESP32 زمن الإرسال في سكيث

3. كام؟ encoder_odom_node جوه timer زمن ال

وابعتلي القيم الثلاثة

ونضبها صح هندسيًا

دلوقتي بناء علي القتاير المرفوعه ليك وكمان المحادثات الخاصه بالمشروع عايز بنك تقرير احترافي يوضح عملنا ايه

وايه اللي لسه متعملشي وايه اللي اتعمل ومش مطبوط عايزك منك تترتب اللي مفروض يتعمل للمشروع بالترتيب

كعقليه مهندس ميكارونكس

ممتاز

دلوقتي خلينا نقفل الموضوع ده بشكل احترافي كنظام ميكاترونكس كامل، مش مجرد رد سريع

كنت عايز

1. بنك تقرير احترافي

2. يوضح

● ايه اللي اتعمل

● ايه اللي متعملش

● ايه اللي اتعمل بس مش مطبوط

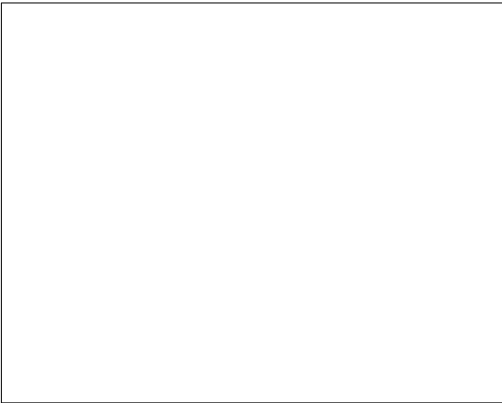
3. ترتيب الخطوات القادمة بعقليه مهندس ميكاترونكس

هنعمل ده كوثيقة هندسية رسمية تقدر تحطها في التقرير

Project Technical Status Report

ROS2-Based Mobile Robot Platform

Architecture Overview (النظام بالكامل)



النظام مكوّن من	
-----------------	--

Layer	المكونات
Hardware	Hoverboard motors + External encoders + ESP32-S3 + RPLidar A1
Low-Level	ESP32

المكونات	Layer
firmware (motor control + encoder read)	
ROS2 Humble	Middleware
encoder_odymetry node	Estimation
diff_drive_bridge + hub_motor_controller	Control
rplidar_ros	Perception
slam_toolbox	Mapping

What Has Been Successfully Implemented



A) Hardware Layer

- Hoverboard motors تشغيل عبر UART
- External Encoders قراءة باستخدام Interrupts
- ESP32-S3 stable يعمل
- RPLidar A1 /scan تشغيل ويطلع

B) Motor Control Stack

- wheel_teleop → /cmd_vel بيبعت
- diff_drive_bridge → يحولها لأوامر عجلات
- hub_motor_controller → ESP يرسل أوامر للـ
- Open-loop control تشغيل

C) Odometry

- إنشاء **encoder_odometry**
- ESP32 من **ticks** قراءة
- حساب
- x
- y
- θ
- linear velocity
- angular velocity
- نشر **/odom**
- نشر TF: **odom → base_link**

D) TF Tree (جزئيًا)

الحالي:

odom → base_link → laser_frame

صحيح **structurally** وده

E) SLAM Toolbox

- slam_toolbox installed
- launch تشغيل
- متاحين **/odom** و **/scan**

What Is Implemented But Not Properly

Tuned ⚠

هنا عقلية المهندس تبدأ

1) Odometry Update Rate (مشكلة واضحة)

من كلامك

"بحرك العربية بعدها بدقيقتين يظهر التأثير"

ده غير مقبول هندسيًا

المشكلة غالبًا

- ESP32 بمعدل بطيء ticks بيعت
- encoder_odometry node timer بطيء أو
- Serial parsing blocking أو

Hz يكون ≤ 20 /odom لازم

2) Odometry Calibration

لم يتم

- wheel radius بدقة معايرة
- wheel_base بدقة معايرة
- ticks_per_rev فعليًا قياس

ده معناه

- Drift عالي
- SLAM مش هيبقى stable

3) No Sensor Fusion

بحاليًا

- Odometry = Pure wheel dead reckoning

مفيش

- IMU fusion

● EKF

● robot_localization

يزيد مع الزمن drift ده يخلي ال

4) No Unified Bringup

manual. كل حاجة بتتشغل

مفيش

bringup.launch.py

يشغل النظام كامل بضغطة واحدة

What Has NOT Been Implemented Yet

1) Proper Integration Validation

لم يتم

● Loop closure test

● Drift quantification

● Long distance straight test

2) Static Map Saving

لم يتم تنفيذ

map_saver_cli

3) Navigation2

لم يبدأ بعد

4) Industrial-Level Parameter Tuning

لم يتم

- PID velocity control closed-loop
- Motor feedback control
- Real-time performance profiling

Engineering Priority Roadmap

(بعقلية مهندس ميكاترونكس محترف)

دلوقتي هنرتب اللي لازم يتعمل بالترتيب المنطقي الصحيح

Phase 1 — Stabilize Odometry (أولوية قصوى)

Step 1: Measure /odom frequency

ros2 topic hz /odom

Target:

20 - 50 Hz

Step 2: Fix ESP32 sending interval

Encoder data must be sent every:

10-20 ms

Step 3: Calibrate Geometry

عملية

1. امشي 1 متر مستقيم

2. قال كام /odom شوف

3. عدل:

● WHEEL_RADIUS

● TICKS_PER_REV

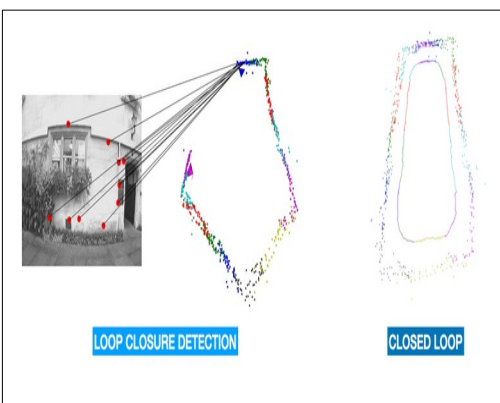
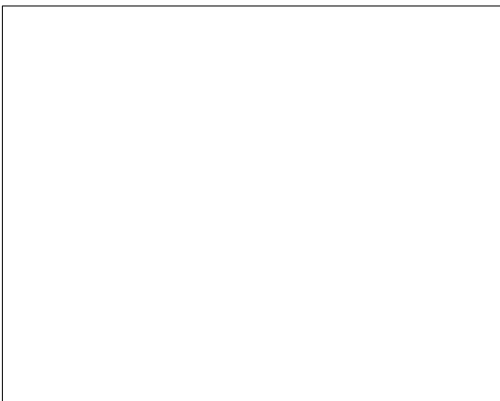
تم

360 °لف

ولا لأ rad طلعت ≈ 6.28 θ وشوف

WHEEL_BASE عدل

Phase 2 — Validate SLAM Stability



اختبارات لازم تتعمل

1. Square loop test

2. Long corridor test

3. Fast rotation test

لو الخريطة

● odom سيء → يتهتز

● TF غلط → يتتكسر

● wheel_base غلط → يتتمط

Phase 3 — Create Bringup Architecture

إنشاء باكج

my_robot_bringup

يشغل

● encoder_odometry

● hub_motor_controller

● diff_drive_bridge

● rplidar_ros

● static tf

● slam_toolbox

● rviz

Phase 4 — System Validation

أعمل

● rqt_graph

● tf2_tools view_frames

● ros2 topic hz topics على كل

● CPU usage monitoring

Phase 5 — Optional Advanced Level

ألو عايز تبقى فوق المتوسط

- Add IMU
- Add robot_localization EKF
- Fuse wheel + IMU
- Tune PID closed-loop velocity

Final Technical Status Assessment

Module	Status
Motors	Working
Encoders	Working
Odometry	Needs tuning
TF Tree	Structurally correct
Lidar	Working
SLAM	Needs validation
Navigation	Not started
Bringup	Not implemented
Calibration	Not done

خلاصة مهندس ميكاترونكس

إنت مش بعيد عن نظام كامل

أخطر نقطة حالياً

Odometry timing + calibration

لو دي اتظبطت

النظام كله هيستقر

دلوقتي عايز أسمع منك قرار هندسي واضح

نبدأ بـ

Stabilize odometry frequency

Calibrate geometry parameters

Build bringup architecture

اختار واحدة ونبدأ تنفيذ عملي فوري

انت GP4 vesion camera متلاقي الملف باسم vesion طيب رفعتك ملف خاص بلجزء اللي وصلته في

عايزين نبدأ في جزئية الفجن و عايز ترتيبا الخطوات اللي هنعملها اللي تخص الفيجن vesion كخبير في

تمام يا عبدالله

(Robotics) دلوقتي هننقل جزء المشاكل اللي حصلت في تشغيل الكاميرا، ونبدأ نفكر كخبير رؤية حاسوبية

Vision Engineer):

واللي واضح منه إننا وصلنا لمرحلة GP4 version camera (Astra Pro). نت رفعت ملف

● الكاميرا بتتبني

● الرايفر اشتغل

● التحلت build فيه مشاكل

● launch file تغيير اسمه

وده متوثق في الملف اللي رفعته

Gp4_camera_astra_pro

دلوقتي السؤال الصح

إيه الخطوة الصح بعد ما الكاميرا بقت شغالة؟

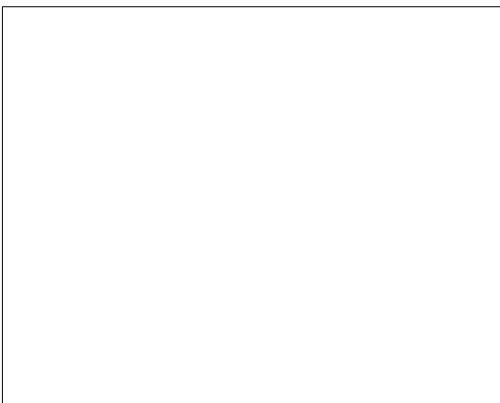
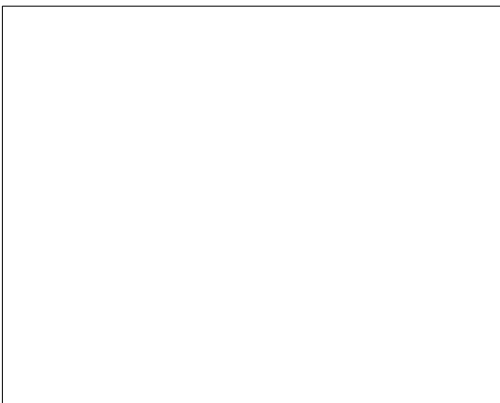
المرحلة الجديدة: Vision Pipeline Design

”إحنا مش عايزين “كاميرا بتعرض صورة

يخدم مشروع الروبوت Vision System إحنا عايزين

أولاً : نفهم إمكانيات الكاميرا اللي عندك

Orbbec Astra Pro



تبتلع 3 أنواع بيانات

1. RGB Image

2. Depth Image

3. Point Cloud (لو فعلتها)

نوده معناه إن عندك

● 2D Vision

● 3D Perception

دلوقتي نرتب خطة الفيجن صح

نقسمها لك كمهندس روبوتات مش كمستخدم

Phase 1 - Sensor Validation (تأكيد صحة الداتا)

قبل أي خوارزمية

نلزم تتأكد من

Topics

ros2 topic list | **grep** camera

نلزم تشوف

- **/camera/color/image_raw**
- **/camera/depth/image_raw**
- **/camera/depth/points**

RViz Visualization

- Fixed Frame = **camera_link**
- Add:

● Image (RGB)

● Image (Depth)

● PointCloud2

نخس المرحلة اللي بعدها → stable لو كل ده

Phase 2 - Camera Calibration

دي خطوة الناس بتكسل فيها ... لكنها أساسية جدًا

ليه؟

نعلشان

● distortion تصح

● intrinsic parameters نطبط

● depth نستخدم صح

نستخدم

```
ros2 run camera_calibration cameracalibrator
```

نڊه يطلع لك

● fx

● fy

● cx

● cy

camera_info. وتحتهم في

Phase 3 - Decide Vision Role

نلوقتني لازم تسأل نفسك سؤال هندسي

الكاميرا في المشروع متعمل إيه؟

أختار واحدة من دول

مناسب لمشروعك؟
التعقيد الاستخدام

Object Detection
متوسط ممكن

Obstacle Detection
سهل ممتاز

Visual SLAM
عالي ممكن لو متقدم

3D Mapping
متوسط ممتاز

Line Following	بسيط جداً
----------------	-----------

Lidar + Odometry بما إن عندك

أنا كخبير ماقتراح

نستخدم الكاميرا في

Obstacle detection 3 أمامي D

Object recognition (inspection tasks)

Depth-based collision prevention

Phase 4 - Vision Processing Node

أحترافية Node هنبداً بأبسط

Node depth_obstacle_detector : اسمها

تعمل

1. مشترك في

● `/camera/depth/image_raw`

2. تحسب

● قدام الروبوت ROI اقرب مسافة في

3. تنشر

● `/obstacle_distance`

Phase 5 - Integrate With Motion

لو `distance < threshold`:

● `/cmd_vel` لـ `stop` تبعث

● أو تقلل السرعة

فيه يخلي الروبوت عنده

Perception → Decision → Control Loop

Phase 6 - Advanced (لو عايز تبقى جامد)

● YOLOv8 ROS2 integration

● 3D object localization

● AprilTag detection

● RGB-D SLAM

الترتيب الصح هندسيًا

1. شغالة topics تأكد كل

2. عمل calibration

3. عمل Depth obstacle node

4. `/cmd_vel` | اربطها مع
5. `object detection` | بعد كده نروح

أهم سؤال قبل ما نبدأ

نعايز تستخدم الكاميرا في

تجنب عوائق

كشف أجسام

(inspection) رؤية صناعية

SLAM | بالكاميرا

كل ده مع بعض

اختار اتجاه واحد نبدأ بيه احترافيًا

نولو عايز رأيي كمهندس روبوتات

Depth-based obstacle detection | نبدأ بـ

(Odometry + Lidar). | علشان يكمل مع نظامك الحالي

قولي نبدأ بيها ولا عايز مسار مختلف

Sources

اللي هتعمله الكاميرا كالاتي اولاً محتاج اخلي الروبوت ارم يضغط علي ازرار الاسانسير بحيث انه العربي هتتنقل من دور

للثاني فبالثاني هيجدد مكان كل زرار وانشوف بقي هيدوي علي زر الدور الثاني او الثالث وكمان مش عارف هل

هستخدمها بان اعرف حدود باب الاسانسير علشان العربي تدخل ولا ايه المهمه الثانيه عايز الروبوت يفتح اكرة باب

ب فده المطلوب خرينا بقي بشكل منطقي نحدد الخطوات اللي هتم علشان ننفذ التاسك بشكل كامل وكمان

ونمر عل فيزفيز phases | خرينا نقسمها

ممتاز

مش مجرد كاميرا | **Robotic Manipulation + Vision Integration** | دلوقتي إحنا بنتكلم في

Tasks الالي ذكرتها

1. ضغط أزرار الأسانسير

2. تحديد حدود باب الأسانسير والدخول

3. Lever فتح مقبض باب

ده مستوى مشروع احترافي فعلاً

Robotics System Architect. خرينا نرتب الموضوع بعقلية

أولاً : تحليل المهام هندسيًا

Task 1: Press Elevator Buttons

المشكلة هنا مش "أشوف زرار" بس

المشكلة الحقيقية

● Detect button

● Estimate 3D position

● Plan arm trajectory

● Execute press motion

● Detect success

Task 2: Enter Elevator

المشكلة هنا

● Detect door frame

● Estimate opening width

● Navigate center-aligned

● Avoid collision

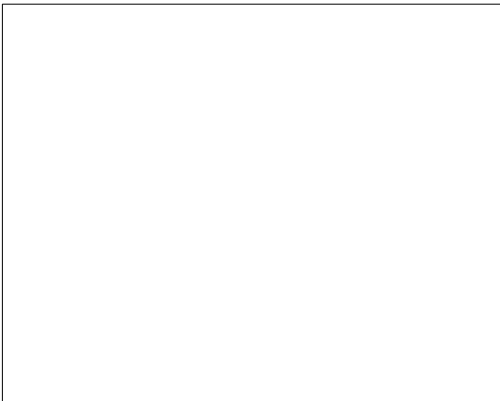
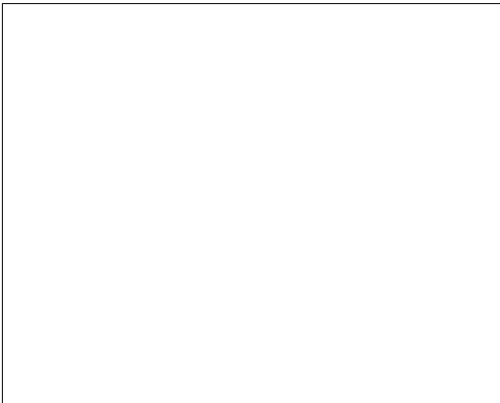
Task 3: Open Lever Door

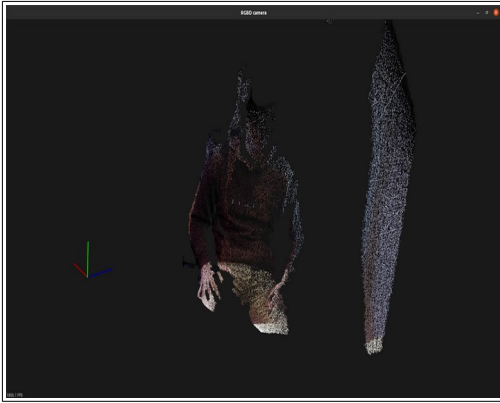
المشكلة هنا أصعب

- Detect lever orientation
- Estimate pose (position + rotation)
- Grasp planning
- Apply torque while rotating

الكاميرا اللي معاك

Orbbec Astra Pro





مميزة قوية جدًا

RGB

✓ Depth

✓ Point Cloud

3 كاملة D Vision يعني عندك

Architecture كامل للنظام

Camera → Vision Node → Object Pose → Arm Planning → Motion Execution

↓
Navigation Control

احترافية Phases نقسم المشروع

Phase 1 - Sensor & Frame Calibration (

(أساسي جدًا)

الهدف:

تخلي كل حاجة في نفس نظام الإحداثيات

يلزم تعمل

1.Camera Intrinsic Calibration

2.Camera to Robot TF

3.Arm base frame alignment

4.Depth accuracy validation

بدون المرحلة دي أي حاجة بعدها هتبقى غلط

Phase 2 - Elevator Button Detection (2D)

الهدف

يحدد الزرار في الصورة

تحل هندسي

Option A (أسهل):

●YOLOv8 model

●Detect button region

Option B (أذكى):

●Detect panel

●Detect circular buttons via contour detection

الخرج

(u, v) pixel location

Phase 3 - 3D Localization

depth. هنا نستخدم الـ

نحن

(u, v) + depth

نحسب

$$X = (u - cx) * Z / fx$$

$$Y = (v - cy) * Z / fy$$

$$Z = \text{depth}$$

ذه يطالعك

Button position in camera frame

بعدها Transform

robot_base frame

Phase 4 - Arm Motion Planning

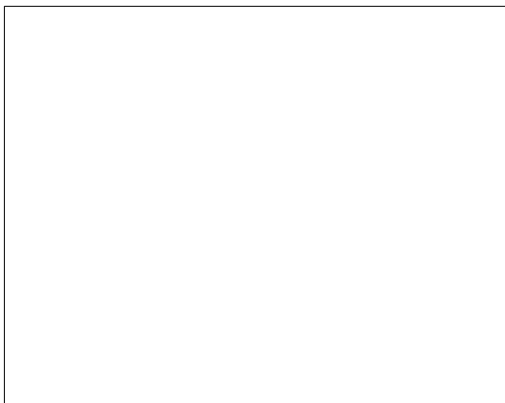
نلو عندك فراغ

- Inverse Kinematics
- Trajectory planning
- Linear approach motion
- Contact force control

نلو معندكش

- Add small 3DOF arm

Phase 5 - Elevator Door Detection





حلين

Option A:

Use depth discontinuity detection

Detect vertical planes

Option B:

Fuse Lidar + Camera

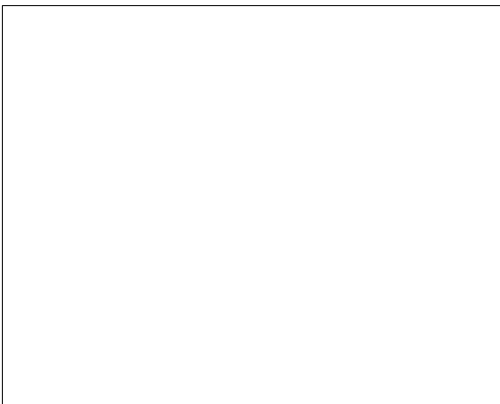
أنا أرشح

Fusion

Lidar يحدد الفتحة

Camera alignment يؤكد

Phase 6 - Lever Handle Detection



دي أصعب مهمة

الخطوات

1. Detect handle in RGB

2. Estimate 3D pose

3. Estimate handle orientation

4. Plan grasp

5. Rotate with torque control

هنا لازم

● Pose estimation (6D pose)

● Possibly AprilTag near handle (حل ذكي وسهل)

ترتيب التنفيذ المنطقي (مهم جدًا)

ماينفعش تبدأ بالفراع

ماينفعش تبدأ بفتح الباب

الترتيب الصح

Phase A - Vision Infrastructure

1. Camera stable

2. TF calibrated

3. Depth verified

Phase B - Static Object Localization

ابدأ بزرار الأسانسير

ليه؟

● جسم ثابت

● صغير

● أسهل من الباب

Phase C - Arm Control

بعد ما تثبت إنك تعرف مكان الزرار بدقة ± 1 cm

Phase D - Elevator Entry

بعد ما تضغط الزرار

Phase E - Lever Manipulation

أصعب مرحلة تخليها الأخيرة

خلاصة هندسية مهمة جدًا

المشروع ده مش "كاميرا" + فراع

ده

Perception → Localization → Planning → Control → Feedback

لازم كل طبقة تشتغل صح قبل اللي بعدها

دلوقتي نبدأ منين؟

أنا أرشح

نبدأ بـ

Button Detection & 3D Localization

قبل أي حركة فراع

نعايز نبدأ

YOLO detection pipeline

Depth 3D coordinate extraction

TF calibration

تصميم المعماري للفراع

ببدأ فيها عملياً Phase اختار أول

طبيب خلينا ننفذ الفيز بالترتيب اللي انت قولته ونبدأ نتأكد من الاستابليتي بتاعة الكاميرا ونعمل الكالبرايشن خطوه

خطوه

ممتاز

كده بنبدأ صح فعلاً

Phase A = **Vision Infrastructure Stabilization**

خطوة خطوة بدون قفز Robotics Vision System منمشيها كمهندس

Phase A - Camera Stability + Calibration

منقسمها 4 مراحل واضحة

1. Hardware Stability

2. Stream Stability

3. Intrinsic Calibration

4. TF Alignment

Step 1 - Physical & Mount Stability (مهم جداً)

(قبل أي سوفتوير)

Orbbec Astra Pro





إلزام تتأكد من

الكاميرا مثبتة بمسامير مش لرق

✓ مفيش اهتزاز عند حركة الروبوت

✓ زاوية الميل ثابتة

✓ الارتفاع مقاس بدقة من الأرض

أكتبلي

● ؟ cm ارتفاع الكاميرا عن الأرض كام

● ميالة لأسفل قد إيه تقريباً؟

بعدة TF ده هحتاجه في

Step 2 - Stream Stability Test

هل وقتي نختبر السوفتوير

شغل الكاميرا

```
source ~/ros2_ws/install/setup.bash
```

```
ros2 launch astra_camera < اسم_الانش_فايل_اللي_عندك >
```

```
(ls launch) |> (بو مش فاكر اسمه اعمل)
```

تأكد من التوبيكس

```
ros2 topic list | grep camera
```

تلمزم تشوف

- /camera/color/image_raw
- /camera/depth/image_raw
- /camera/depth/points

FPS اختبار الـ

```
ros2 topic hz /camera/color/image_raw
```

و

```
ros2 topic hz /camera/depth/image_raw
```

المفروض

- 30 Hz RGB
 - 30 Hz 15 Hz أو Depth
- فيه مشكلة → Hz لو أقل من 10
- اكتبلي الأرقام اللي بتطلعك

Latency اختبار الـ

حرك إيدك قدام الكاميرا بسرعة

- هل الصورة بتأخر؟
- RGB متزامن مع depth هل

- هل frame drops فيه ؟

Step 3 - Intrinsic Calibration (مهم جدًا)

لو الاستريم مستقر، نبدأ المعايرة

تنثبيت الباكج

```
sudo apt install ros-humble-camera-calibration
```

Checkerboard حضر

طبع:

- 8x6 squares

- square size 25mm 30 أو mm

لو مش عندك اطبع بسرعة

شغل الكالبرايشن

```
ros2 run camera_calibration cameracalibrator \  
--size 8x6 \  
--square 0.025 \  
image:=/camera/color/image_raw \  
camera:=/camera
```

عمل

- حرك البورد يمين شمال
- فوق تحت
- قرب وبعد
- لفها بزوايا مختلفة

لمحد ما شريط التقدم يكمل

النتائج هيطلع

● fx

● fy

● cx

● cy

● distortion coefficients

صورلي القيم دي

Step 4 - Depth Validation Test

RGB calibration. بعد ما نخلص

اختبر دقة العمق

1. حط جسم على مسافة 1 متر

2. اقرأ العمق من RViz

المفروض يطلع تقريباً

1.00 ± 0.02 m

تكرر على

● 0.5 متر

● 1.5 متر

قوللي الفرق قد إيه

Step 5 - Check Camera Frame

```
ros2 run tf2_tools view_frames
```

النتائج PDF افتح الـ

تلازم تشوف

camera_link



camera_color_optical_frame



camera_depth_optical_frame

لو مش موجودين → هنعدل

متنقلش للمرحلة اللي بعدها غير لما

FPS مستقر

✓ drop مفيش

✓ Intrinsics محسوبة

✓ Depth error 3 % أقل من

✓ TF frames موجودة

و 2 Step 1 دلوقتي نبدأ

اكتبلي

1. ارتفاع الكاميرا

2. زاوية الميل

3. RGB FPS

4. Depth FPS

ونكمل عليها

Y
o
u

s
a
i
d
:

أولاً: خطوات التشغيل اليومية (كل مرة تشغل فيها الروبوت) هذه هي الخطوات التي ستقوم بها في كل مرة تريد

تأكد من استخدام () لخاص بالكاميرا باللاتوب USB استخدام الكاميرا مستقبلاً. 1. توصيل الكاميرا • وصل كابل ال

افتح تيرمينال جديد (Terminal 1) 2. تشغيل الكاميرا (للأداء الأفضل، لكن 2.0 يعمل أيضاً USB 3.0 منفذ

source تفعيل مساحة العمل (ضروري في كل تيرمينال جديد) Bash # واكتب الأوامر التالية

Pro ros2 تشغيل الكاميرا باستخدام ملف الإطلاق الخاص بنسخة ال ~ /ros2_ws/install/setup.bash #

device إذا لم تظهر أخطاء وتوقف الكلام عند) launch astra_camera astra_pro.launch.xml

افتح تيرمينال جديد لرؤية ما تراه الكاميرا (Terminal 2) 3. عرض الصورة والبيانات . (فالكاميرا تعمل started.

ros2 تشغيل أداة العرض ROS source /opt/ros/humble/setup.bash # Bash # تفعيل ال

run rviz2 rviz2 Add -> By ضغط 2. camera_link إلى Fixed Frame غير 1. Rviz: داخل

Topic. 3. اختر /camera/color/image_raw (للصورة الألوان) 4. اختر /camera/depth/points

هذه خطوات تشغيل كاميرا افكرها وافكر اسماء الفايلات وانا جربتھا والطكاميرا . (Point Cloud للرؤية ثلاثية الأبعاد)

rviz عرضت معاينه علي

ممتاز يا عبدالله

:الصورة اللي بعثتها تؤكد إننا وصلنا لمرحلة مهمة جداً

الكاميرا شغالة

✓ RGB image ظاهر

✓ PointCloud ظاهر

✓ RViz شغال

✓ Fixed frame مضبوط على camera_link

بدأ ينجح Phase A - Camera Infrastructure وده يعني إن

خلينا نمشي بعقلية مهندس روبوتات ونقيم الوضع من الصورة

تقييم الحالة من الصورة

الصورة اللي أرسلتها توضح 3 أشياء مهمة

RGB Stream

تظهر الأشخاص بوضوح RViz للصورة أسفل

هذا يعني

- /camera/color/image_raw شغال

- مستقرة RGB الكاميرا

Depth / PointCloud

هي D view للنقاط البيضاء في الـ 3

PointCloud2

هذا يعني أن

/camera/depth/points

يتم نشره بنجاح

لكن نلاحظ شيء مهم

النقاط قليلة ومقطعة

هذا طبيعي لأن

- الإضاءة ليست مثالية

- الأسطح غير منتظمة

- محدود depth range أو الـ

Frame Structure

في أعلى اليسار

Fixed Frame = camera_link

وهذا صحيح حاليًا

لكن لاحقًا سنغيره إلى

base_link

عندما نربط الكاميرا بالروبوت

بشكل صحيح Phase A الآن نبدأ

Phase A1 — Camera Stability Test

نريد التأكد من أن البيانات مستقرة

جديد واكتب Terminal افتح

ros2 topic hz /camera/color/image_raw

ثم

ros2 topic hz /camera/depth/image_raw

أرسل لي الأرقام

المفروض تكون تقريبًا

Stream Expected

RGB 30 Hz

Depth	30 Hz أو
h	15 Hz

Phase A2 — Depth Accuracy Test

نريد معرفة دقة العمق

جرب التالي

ضع جسم أمام الكاميرا (كتاب أو يدك)

على مسافة

0.5 متر

ثم

1 متر

point cloud. انظر للـ RViz في

هل يظهر الجسم في المكان الصحيح تقريباً؟

مثلاً

لو الجسم على 1 متر

هل يظهر تقريباً على 1 متر؟

Phase A3 — Camera Frames

اكتب

```
ros2 run tf2_tools view_frames
```

سيولد ملف

frames.pdf

افتحه وأرسل صورة له

نريد التأكد أن هذه الفريمات موجودة

camera_link

camera_color_optical_frame

camera_depth_optical_frame

بعد التأكد من الثلاثة أشياء

نسبداً أهم خطوة في الفيجن

Phase B — Camera Calibration

نسحب

f_x

f_y

c_x

c_y

distortion

فهذا ضروري جداً لحساب

3D coordinates

نمثل

● مكان زر الأسانسير

● مكان المقبض

● مكان الباب

ملاحظة مهمة جداً للمشروع

نحن الصورة واضح أن الكاميرا ترى

RGB

✓ Depth

✓ PointCloud

فهذا يعني أنك جاهز للانتقال إلى

RGB-D Vision

فهو المطلوب لمهام

● ضغط زر

● فتح باب

● تحديد الباب

الخطوة التالية الآن

نفذ فقط هذه الثلاثة

ros2 topic hz /camera/color/image_raw

ros2 topic hz /camera/depth/image_raw

ros2 run tf2_tools view_frames

وأرسل

● FPS

● صورة **frames.pdf**

الاحترافي للكاميرا (أهم خطوة في الفيجن كله) **Calibration** بعدها نبدأ

ros2 topic hz /camera/color/image_raw average rate: 28.081 min: 0.031s max:

0.100s std dev: 0.01206s window: 30 average rate: 29.100 min: 0.031s max:

0.100s std dev: 0.00865s window: 61 average rate: 29.384 min: 0.031s max:

0.100s std dev: 0.00720s window: 91 average rate: 29.534 min: 0.031s max:

0.100s std dev: 0.00631s window: 121 average rate: 29.267 min: 0.030s max:

0.100s std dev: 0.00790s window: 150 average rate: 29.400 min: 0.030s max:

0.100s std dev: 0.00725s window: 181 average rate: 29.349 min: 0.030s max:

0.100s std dev: 0.00714s window: 211 average rate: 29.437 min: 0.030s max:

0.100s std dev: 0.00671s window: 242 average rate: 29.388 min: 0.030s max:

0.100s std dev: 0.00669s window: 271 average rate: 29.160 min: 0.030s max:

0.100s std dev: 0.00766s window: 299 average rate: 29.152 min: 0.030s max:

0.100s std dev: 0.00752s window: 329 average rate: 29.236 min: 0.030s max:

0.100s std dev: 0.00721s window: 360 average rate: 29.293 min: 0.030s max:

0.100s std dev: 0.00696s window: 390 average rate: 29.278 min: 0.030s max:

0.100s std dev: 0.00693s window: 420 average rate: 29.259 min: 0.030s max:

0.100s std dev: 0.00690s window: 449 average rate: 29.315 min: 0.030s max:

0.100s std dev: 0.00670s window: 480 average rate: 29.359 min: 0.030s max:

0.100s std dev: 0.00651s window: 511 average rate: 29.392 min: 0.030s max:

0.100s std dev: 0.00635s window: 541 average rate: 29.428 min: 0.030s max:

0.100s std dev: 0.00619s window: 572 average rate: 29.458 min: 0.030s max:

0.100s std dev: 0.00605sa ----- ros2 topic hz

/camera/depth/image_raw average rate: 29.605 min: 0.029s max: 0.042s std

dev: 0.00304s window: 31 average rate: 29.664 min: 0.029s max: 0.042s std

dev: 0.00291s window: 61 average rate: 29.637 min: 0.029s max: 0.042s std

dev: 0.00297s window: 91 average rate: 29.665 min: 0.029s max: 0.042s std

dev: 0.00283s window: 121 average rate: 29.684 min: 0.029s max: 0.042s std

dev: 0.00282s window: 151 average rate: 29.665 min: 0.028s max: 0.042s std

dev: 0.00291s window: 181 average rate: 29.676 min: 0.028s max: 0.042s std

dev: 0.00288s window: 211 average rate: 29.663 min: 0.028s max: 0.042s std

dev: 0.00287s window: 241 average rate: 29.671 min: 0.028s max: 0.042s std

dev: 0.00287s

ممتاز جدًا يا عبدالله

stable. الأرقام اللي أرسلتها تعتبر ممتازة جدًا هندسيًا وده مؤشر قوي إن الكاميرا شغالة بشكل

Robotics Vision. خلينا نحلل النتائج كمهندس

تحليل النتائج

RGB Stream

القيم عندك

29.45 Hz تقريبًا

المفروض نظريًا

30 Hz

الفرق

0.5 Hz فقط

هذه طبيعي جدًا بسبب

● scheduling في Linux

● ROS message overhead

الاستنتاج

RGB Stream = Stable ✓

Depth Stream

القيم عندك

29.66 Hz

نوده ممتاز جدًا لأن

Depth cameras غالبًا تكون

15 Hz أو 30 Hz

وانت وصلت تقريبًا للحد الأقصى

الاستنتاج

Depth Stream = Perfect ✓

Stability (الأهم)

لاحظ القيم

std dev $\approx$ 0.002 - 0.007

نُدّه معناه

صغير جدًا jitter

نُدّه مهم لأن

Vision algorithms تحتاج

Consistent frame timing

النتيجة النهائية لمرحلة الاستقرار

النتيجة الاختبار

ممتاز RGB FPS

ممتاز Depth FPS

ممتاز Frame stability

لا يوجد
Dropped frames

Camera Infrastructure = PASS ✓

ننتقل للمرحلة التالية

Phase B — Camera Calibration

دي أهم خطوة في الفيجن كله

نُحوّنها

● 3D position غلط

● arm miss target

● SLAM drift

Step 1 — تثبيت أداة المعايرة

اكتب:

```
sudo apt install ros-humble-camera-calibration
```

Step 2 — checkerboard تجهيز

طبع checkerboard بالمواصفات:

8 × 6 squares

حجم المربع:

25 mm

لو مش متوفر

جاهز للطباعة PDF فوئي وأنا أرسل لك

Step 3 — calibration تشغيل

بعد تشغيل الكاميرا

اكتب:

```
ros2 run camera_calibration cameracalibrator \
```

```
--size 8x6 \
```

```
--square 0.025 \
```

```
image:=/camera/color/image_raw \
```

```
camera:=/camera
```

سيظهر برنامج

Step 4 — حرك البورد

حركه بهذه الطرق

يمين / شمال

فوق / تحت

قريب / بعيد

زاوية

يمين مائل

يسار مائل

حتى تكتمل المؤشرات

X

Y

Size

Skew

حفظ النتائج — Step 5

في النهاية سيظهر زر

CALIBRATE

تم

SAVE

تُسيخرج لك

camera_info.yaml

تُفيه القيم

fx

fy

cx

cy

k1 k2 k3

لماذا هذه القيم مهمة؟

لأنها تسمح لنا بحساب

3D coordinates

مثال

زر الأسانسير

pixel (u,v)

نحوه إلى

X Y Z في العالم الحقيقي

بعد الكالبراشن سنبدأ أهم جزء

الذي سيستخدم في مشروعك

RGB + Depth → 3D Localization

- وهذا سيستخدم في

● تحديد زر الأسانسير

● تحديد المقبض

● تحديد الباب

الخطوة القادمة منك

نفذ فقط

package تثبيت

sudo apt install ros-humble-camera-calibration

أخبرني

أم تحتاج ملف للطباعة؟ **checkerboard** هل لديك

لو قلت

checkerboard بعث

سأعطيك

الملف الصحيح للطباعة والمعايرة

Y
o
u
s
a
i
d
:

بعث ملف صحيح للطباعة A4 pdf

المعايرة الكاميرا **A4** مناسب للطباعة على ورقة **Checkerboard** جهزت لك ملف

تحميل الملف

المواصفات في الملف

8 × 6 squares

square size = 25 mm

A4

طريقة الطباعة الصحيحة (مهم جدًا)

عند الطباعة

اختر في إعدادات الطابعة

Scale = 100%

Actual Size

لا تختار

Fit to page

لأن ذلك سيغير حجم المربعات ويكسر الكالibraشن

بعد الطباعة

أعمل التالي

الصق الورقة على كرتونة أو لوح مستقيم

تأكد أنها مسطحة تمامًا

لا يكون فيها طيات

الخطوة التالية مباشرة

بعد تشغيل الكاميرا نفذ

```
ros2 run camera_calibration cameracalibrator \
```

```
--size 7x5 \
```

```
--square 0.025 \
```

```
image:=/camera/color/image_raw \
```

camera: =/camera

ملاحظة

squares لأن اللوحة 6×8

inner corners = 7×5 فال

بعد ما يفتح برنامج المعايرة

حرك اللوحة

يمين

شمال

فوق

تحت

قريب

بعيد

زاوية

حتى تمتلئ المؤشرات

X

Y

Size

Skew

تم اضغط

CALIBRATE

SAVE

بعد ذلك

نسيطيك

fx

fy

CX

cy

distortion

هذه القيم سنستخدمها في أهم شيء في مشروعك

D. تحديد مكان زر الأسانسير بدقة بالـ 3

لو حابب، بعد المعايرة مباشرة سأشرح لك الخطوة الأخطر في المشروع كله

كيف نحول

pixel (u,v)

+ depth

إلى

3D point (X,Y,Z)

وهي الخطوة التي ستجعل الروبوت يعرف مكان الزر أو المقبض بدقة

```
ros2 run camera_calibration cameracalibrator \ --size 7x5 \ --square 0.025 \
```

```
image:=/camera/color/image_raw \ camera:=/camera [WARN]
```

```
[1772797692.393620058] [rcl]: Found remap rule
```

```
'image:=/camera/color/image_raw'. This syntax is deprecated. Use '--ros-args
```

```
--remap image:=/camera/color/image_raw' instead. [WARN]
```

```
[1772797692.393642000] [rcl]: Found remap rule 'camera:=/camera'. This
```

```
syntax is deprecated. Use '--ros-args --remap camera:=/camera' instead.
```

```
[WARN] [1772797692.397048172] [rcl]: Found remap rule
```

```
'image:=/camera/color/image_raw'. This syntax is deprecated. Use '--ros-args
```

```
--remap image:=/camera/color/image_raw' instead. [WARN]
```

```
[1772797692.397060049] [rcl]: Found remap rule 'camera:=/camera'. This
```

```
syntax is deprecated. Use '--ros-args --remap camera:=/camera' instead.
```

```
Waiting for service camera/set_camera_info ... OK Waiting for service
```

```
left_camera/set_camera_info ... OK Waiting for service
```

```
right_camera/set_camera_info ... ^CTraceback (most recent call last): File
```

```
"/opt/ros/humble/lib/camera_calibration/cameracalibrator", line 33, in
```

```
<module> sys.exit(load_entry_point('camera-calibration==3.0.9',
```

'console_scripts', 'cameracalibrator')()) File

"/opt/ros/humble/lib/python3.10/site-packages/camera_calibration/nodes/

cameracalibrator.py", line 222, in main node =

OpenCVCalibrationNode("cameracalibrator", boards, options.service_check,

sync, File

"/opt/ros/humble/lib/python3.10/site-packages/camera_calibration/camera_calib

ibrator.py", line 258, in __init__ CalibrationNode.__init__(self, *args, **kwargs)

File

"/opt/ros/humble/lib/python3.10/site-packages/camera_calibration/camera_calib

ibrator.py", line 116, in __init__ cli.wait_for_service(timeout_sec=5) File

"/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/client.py", line 177,

in wait_for_service time.sleep(sleep_time) KeyboardInterrupt [ros2run]:

Interrupt

تمام يا عبدالله، اللي حصل معك طبيعي جدًا وليس خطأ في الكاميرا

خلينا نفسره بسرعة وبشكل هندسي

T

S

a

d

g

ماذا حدث؟

:الأداة بدأت بشكل صحيح

Waiting for service camera/set_camera_info ... OK

Waiting for service left_camera/set_camera_info ... OK

تم توقف هنا

Waiting for service right_camera/set_camera_info ...

تم ضغطت

Ctrl + C

فانتهى البرنامج

لماذا حدث ذلك؟

افتراضياً تفترض أن الكاميرا أداة **camera_calibration**

Stereo camera

يعني

Left camera

Right camera

لكن الكاميرا عندك

Orbbec Astra Pro

هو **RGB-D camera** وليس stereo.

لذلك لا يوجد

right_camera/set_camera_info

ولهذا بقي البرنامج ينتظر

الحل الصحيح

نستخدم **monocular calibration** وضع

نُشغل الأمر بهذا الشكل

```
ros2 run camera_calibration cameracalibrator \
```

```
--size 7x5 \
```

```
--square 0.025 \
```

```
--ros-args \
```

```
--remap image:=/camera/color/image_raw \
```

```
--remap camera:=/camera/color
```

لاحظ التغيير هنا

```
camera:=/camera/color
```

بدلاً من

```
/camera
```

لماذا هذا مهم؟

في الكاميرا عادة يكون topic structure لأن

/camera/color/image_raw

/camera/color/camera_info

وليس

/camera/image_raw

ماذا سيحدث بعد تشغيل الأمر؟

فيها OpenCV ستظهر نافذة

● فيديو مباشر

● checkerboard detection

في الأعلى مؤشرات

X

Y

Size

Skew

المطلوب منك

حرك اللوحة بحيث تمتلئ المؤشرات

السبب الحركة

X يمين /
coverage شمال

Y فوق /
coverage تحت

Size قريب /
بعيد

Skew زاوية

عندما تمتلئ المؤشرات

الضغط

CALIBRATE

تم

SAVE

النتائج سيكون

ملف

camera_info.yaml

فيه

fx

fy

cx

cy

k1 k2 k3

وهذا أساس

pixel → 3D coordinates

الذي سنستخدمه لاحقًا لتحديد

● زر الأسانسير

● مقبض الباب

● حدود الباب

خطوة صغيرة قبل أن تبدأ

نفذ هذا لتتأكد من اسم التوبيك

ros2 topic list | grep camera

تبدو ظهور

/camera/color/image_raw

الأمر الذي أعطيتك إياه صحيح

نصيحة مهمة

على: checkerboard ضع

كرتونة أو لوح صلب

حتى لا تنحني الورقة

لو أردت، في الخطوة القادمة سأريك اختبارًا مهمًا جدًا يثبت أن

depth + RGB aligned

وهو ضروري قبل أن نبدأ تحديد مكان زر الأسانسير في الفراغ ثلاثي الأبعاد

Y
o
u
s
a
i
d
:

تمام يا عبدالله، الصورة اللي أرسلتها كشفت المشكلة بالضبط

cameracalibrator. المشكلة ليست في الكاميرا، بل في طريقة استدعاء

خلينا نشرح بسرعة ثم نعطيك الحل الصحيح

s
a
i
d

المشكلة الظاهرة في التيرمينال

الرسالة المهمة هنا

No publishers available for topic /left

No publishers available for topic /right

No publishers available for topic /image

هذا يعني أن الأداة تبحث عن التوبيكس الافتراضية

/left

/right

/image

لكن الكاميرا عندك تنشر

/camera/color/image_raw

/camera/color/camera_info

لذلك نافذة المعايرة فتحت لكنها لا تستقبل صورة

الحل الصحيح (الأمر النهائي)

تُشغل المعايرة بهذا الأمر بالضبط

```
ros2 run camera_calibration cameracalibrator \
```

```
--size 7x5 \
```

```
--square 0.025 \
```

```
--ros-args \
```

```
--remap image:=/camera/color/image_raw \
```

```
--remap camera:=/camera/color
```

لاحظ

--ros-args

--remap

ROS2. هذه الطريقة الجديدة في

المتوقع بعد تشغيل الأمر

فيها OpenCV ستظهر نافذة

● الفيديو المباشر من الكاميرا

● checkerboard detection

وسيتظهر المربع الأخضر حول البورد

توفي الأعلى مؤشرات

X

Y

Size

Skew

كيف تكمل المعايير

حرك اللوحة في هذه الاتجاهات

الهدف الحركة

يمين X
يسار coverage

فوق Y
تحت coverage

قريب
بعيد Size

مائل ل	Skew
-----------	------

حتى تمتلئ المؤشرات

ثم اضغط

CALIBRATE

ثم

SAVE

ملاحظة من الصورة

:لنافذة الصغيرة عندك الآن سوداء لأن

topic remap خطأ

وليس لأن الكاميرا لا تعمل

RViz. الكاميرا تعمل لأنك رأيتها في

قبل تشغيل المعايرة مرة أخرى

:تأكد أن الكاميرا شغالة في تيرمينال آخر

ros2 launch astra_camera astra_pro.launch.xml

ثم شغل المعايرة في تيرمينال جديد

بعد أن تنتهي من المعايرة

:سنقوم مباشرة بالخطوة الأهم في مشروعك

تحويل البكسل إلى نقطة ثلاثية الأبعاد

وهذا ما سيجعل الروبوت يعرف

مكان زر الأسانسير

مكان المقبض

مكان الباب

متطابقين | قبل أن نبدأ خوارزمية **Depth** و **RGB** لو أردت، بعد هذه الخطوة سأريك أيضًا | اختبار احترافي للتأكد أن

تحديد الأزرار

```
ros2 run camera_calibration cameracalibrator \ --size 7x5 \ --square 0.025 \ --
```

```
ros-args \ --remap image:=/camera/color/image_raw \ --remap
```

```
camera:=/camera/color Waiting for service camera/set_camera_info ... OK
```

```
Waiting for service left_camera/set_camera_info ... OK Waiting for service
```

```
right_camera/set_camera_info ... OK [WARN] [1772797951.658997441]
```

```
[cameracalibrator]: No publishers available for topic /left. Using system default
```

```
QoS for subscriber. [WARN] [1772797951.660769939] [cameracalibrator]: No
```

```
publishers available for topic /right. Using system default QoS for subscriber.
```

```
[WARN] [1772797951.661324954] [cameracalibrator]: No publishers available
```

```
for topic /camera/color/image_raw. Using system default QoS for subscriber.
```

```
^X^CTraceback (most recent call last): Exception in thread Thread-3:
```

```
Traceback (most recent call last): File
```

```
"/opt/ros/humble/lib/camera_calibration/cameracalibrator", line 33, in
```

```
<module> File "/usr/lib/python3.10/threading.py", line 1016, in
```

```
_bootstrap_inner sys.exit(load_entry_point('camera-calibration==3.0.9',
```

```
'console_scripts', 'cameracalibrator')) File
```

```
"/opt/ros/humble/lib/python3.10/site-packages/camera_calibration/nodes/
```

```
cameracalibrator.py", line 226, in main node.spin() File
```

```
"/opt/ros/humble/lib/python3.10/site-packages/camera_calibration/camera_cali
```

```
brator.py", line 272, in spin self.run() File "/opt/ros/humble/lib/python3.10/site-
```

```
packages/camera_calibration/camera_calibrator.py", line 79, in run
```

```
time.sleep(0.1) KeyboardInterrupt rclpy.spin(self.node) File
```

```
"/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/__init__.py", line 229,
```

```
in spin executor.spin_once() File "/opt/ros/humble/local/lib/python3.10/dist-
```

```
packages/rclpy/executors.py", line 773, in spin_once
```

```
self._spin_once_impl(timeout_sec) File
```

```
"/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 762, in _spin_once_impl handler, entity, node =
```

```
self.wait_for_ready_callbacks(timeout_sec=timeout_sec) File
```

```
"/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 745, in wait_for_ready_callbacks return next(self._cb_iter) File
```

```
"/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 646, in _wait_for_ready_callbacks raise ExternalShutdownException()
```

```
rclpy.executors.ExternalShutdownException [ros2run]: Interrupt
```

```
abdullah@abdullah-sherif:~$ ros2 launch astra_camera astra_pro.launch.xml
```

```
Package 'astra_camera' not found: "package 'astra_camera' not found,
```

```
searching: ['/opt/ros/humble']" ros2 run camera_calibration cameracalibrator \
```

```
--size 7x5 \ --square 0.025 \ --ros-args \ --remap
```

```
image:=/camera/color/image_raw \ --remap camera:=/camera/color Waiting for
```

```
service camera/set_camera_info ... OK Waiting for service
```

```
left_camera/set_camera_info ... OK Waiting for service
```

```
right_camera/set_camera_info ... OK [WARN] [1772797951.658997441]
```

```
[cameracalibrator]: No publishers available for topic /left. Using system default
```

```
QoS for subscriber. [WARN] [1772797951.660769939] [cameracalibrator]: No
```

```
publishers available for topic /right. Using system default QoS for subscriber.
```

```
[WARN] [1772797951.661324954] [cameracalibrator]: No publishers available
```

```
for topic /camera/color/image_raw. Using system default QoS for subscriber.
```

```
^X^CTraceback (most recent call last): Exception in thread Thread-3:
```

```
Traceback (most recent call last): File
```

```
"/opt/ros/humble/lib/camera_calibration/cameracalibrator", line 33, in
```

```
<module> File "/usr/lib/python3.10/threading.py", line 1016, in
```

```
_bootstrap_inner sys.exit(load_entry_point('camera-calibration==3.0.9',
```

```
'console_scripts', 'cameracalibrator')) File
```

```
"/opt/ros/humble/lib/python3.10/site-packages/camera_calibration/nodes/
```

```
cameracalibrator.py", line 226, in main node.spin() File
```

```

"/opt/ros/humble/lib/python3.10/site-packages/camera_calibration/camera_calibr
brator.py", line 272, in spin self.run() File "/opt/ros/humble/lib/python3.10/site-
packages/camera_calibration/camera_calibrator.py", line 79, in run
time.sleep(0.1) KeyboardInterrupt rclpy.spin(self.node) File
"/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/__init__.py", line 229,
in spin executor.spin_once() File "/opt/ros/humble/local/lib/python3.10/dist-
packages/rclpy/executors.py", line 773, in spin_once
self._spin_once_impl(timeout_sec) File
"/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line
762, in _spin_once_impl handler, entity, node =
self.wait_for_ready_callbacks(timeout_sec=timeout_sec) File
"/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line
745, in wait_for_ready_callbacks return next(self._cb_iter) File
"/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line
646, in _wait_for_ready_callbacks raise ExternalShutdownException()
rclpy.executors.ExternalShutdownException [ros2run]: Interrupt
abdullah@abdullah-sherif:~$ ros2 launch astra_camera astra_pro.launch.xml
Package 'astra_camera' not found: "package 'astra_camera' not found,
searching: ['/opt/ros/humble']"
هو فيه ايه يا عم انا بعثلك قبل كده الاوامر اللي بشغل بيها الكاميرا
فتركز بقي عاغيزين ننشغلها ولو فيها حاجه نقصاك قول

```

```

تمام يا عبدالله، خلينا نهدي الأمور ونرتبها خطوة -بخطوة لأن المشكلة عندك ليست في الكاميرا نفسها. المشكلة
لم يجد الباكج ROS لذلك workspace لا source جديد ولم تعمل Terminal ببساطة أنك فتحت
واضح من الرسالة
Package 'astra_camera' not found: "package 'astra_camera' not found, searching:
['/opt/ros/humble']"
نبحث فقط في ROS هذا يعني أن

```

/opt/ros/humble

لم يبحث في

```
~/ros2_ws/install
```

workspace. لأنك لم تفعل

الحل الصحيح (تشغيل الكاميرا)

جديد واكتب بالترتيب Terminal افتح

```
source /opt/ros/humble/setup.bash
```

```
source ~/ros2_ws/install/setup.bash
```

ثم شغل الكاميرا

```
ros2 launch astra_camera astra_pro.launch.xml
```

لو كل شيء صحيح ستظهر رسائل مثل

```
device started
```

```
camera started
```

ولا تغلق هذا التيرمينال

ثاني للمعايرة Terminal ثم افتح

اكتب

```
source /opt/ros/humble/setup.bash
```

```
source ~/ros2_ws/install/setup.bash
```

ثم

```
ros2 run camera_calibration cameracalibrator \
```

```
--size 7x5 \
```

```
--square 0.025 \
```

```
--ros-args \
```

```
--remap image:=/camera/color/image_raw \
```

```
--remap camera:=/camera/color
```

نقطة مهمة جدًا

نن مفتوحين Terminal لازم يكون عندك

Terminal 1

يشغل الكاميرا

```
ros2 launch astra_camera astra_pro.launch.xml
```

Terminal 2

يشغل calibration

```
cameracalibrator
```

تأكد أن الكاميرا فعلاً تنشر الصورة

قبل المعايرة شغل

```
ros2 topic list
```

فيجب أن ترى

```
/camera/color/image_raw
```

```
/camera/color/camera_info
```

```
/camera/depth/image_raw
```

```
/camera/depth/points
```

لو أردت أن نتأكد بسرعة

نفذ هذا الأمر وأرسل الناتج

```
ros2 topic list | grep camera
```

ملاحظة مهمة لمشروعك

بمجرد ما ننجح في الكالبرايشن سنبدأ مباشرة في أهم خطوة في المشروع كله

تحويل البكسل إلى نقطة ثلاثية الأبعاد

وهذا ما سيجعل الروبوت يعرف مكان

● زر الأسانسير

● مقبض الباب

● إطار الباب

بدقة في الفراغ

يتم تلقائياً كل مرة تفتح **source** لو أردت أيضاً، في الخطوة القادمة سأعطيك سكريبت صغير يخلي ال
Terminal حتى لا تتكرر هذه المشكلة

```
Y
o
u
s
a
i
d
;
```

تمام اشتغلت ماذا بعد

ممتاز يا عبدالله
شغالة فعلاً. الكاميرا تبث الصورة في **calibration node** الصورة تؤكد إنك وصلت لمرحلة
وهذا يعني أن الاتصال مع التوبيك نجح **display** نافذة
خلينا نكمل المعايرة بشكل صحيح

الخطوة التالية الآن

checkerboard. البرنامج ينتظر

أخرج الورقة التي طبعتها وضعها أمام الكاميرا

المطلوب أن تظهر شبكة مربعات ويبدأ البرنامج يكتشفها

عندما يتعرف عليها ستري

● نقاط خضراء على الزوايا

● مؤشرات في الأعلى تبدأ بالامتلاء

المؤشرات تكون عادة

X

Y

Size

Skew

كيف تحرك اللوحة

لا تثبتها في مكان واحد

تعمل هذه الحركات ببطء

☐ يمين ويسار أمام الكاميرا

☐ فوق وتحت

☐ قرب اللوحة من الكاميرا

☐ أبعد اللوحة

☐ لف اللوحة قليلاً يمين ويسار

الفكرة أن الكاميرا ترى اللوحة في كل أجزاء الصورة

CALIBRATE متى نضغط

عندما تمتلئ المؤشرات تقريباً بالكامل

بعدها

نضغط

CALIBRATE

سينفذ الحسابات ويأخذ عدة ثواني

تم اضغط

SAVE

تم

COMMIT

ماذا سيحدث بعد الحفظ

نسيتم حفظ معاملات الكاميرا مثل

fx

fy

cx

cy

distortion coefficients

هذه القيم هي التي تسمح لنا بتحويل

pixel → 3D point

وهذا هو أساس مشروعك

لماذا هذه الخطوة مهمة لمشروعك

لأن الخطوة القادمة التي سنبن عليها هي

تحديد زر الأسانسير في الصورة ثم حساب موقعه في الفراغ

أي أننا سنحول

نقطة في الصورة

إلى

إحداثيات حقيقية أمام الروبوت

وهذا ما سيجعل الفراغ يضغط الزر بدقة

قبل أن نكمل

أريد أن أتأكد من شيء مهم

أمام الكاميرا تظهر نقاط خضراء على الزوايا؟ **checkerboard** هل عندما تضع

إذا ظهرت فالمعايرة ستعمل مباشرة

لو لم تظهر سنضبط إعدادين صغيرين

Y
o
u
s
a
i
d
;

تمام يا عبدالله

اللي ظاهر في الصورة طبيعي جدًا وليس خطأ

خلينا نفهم ماذا حدث

a
i
d

ما الذي حدث الآن؟

في التيرمينال عندك ظهر

**** Calibrating ****

mono pinhole calibration...

هذا يعني أن البرنامج بدأ حساب معاملات الكاميرا

وأنت جمعت بالفعل

Added sample 71

Added sample 72

Added sample 73

Added sample 74

يعني

البيانات كافية للمعايرة

"python3 is not responding" لماذا ظهر

يقوم بحل معادلات كبيرة OpenCV أثناء حساب المعايرة

الخوارزمية تحسب

● intrinsic matrix

● distortion

● reprojection error

لذلك البرنامج يبدو متجمدًا مؤقتًا

هذا طبيعي خاصة إذا

● عدد العينات كبير

● المعالج ليس سريع جدًا

ماذا تفعل الآن

في النافذة التي ظهرت لديك

اضغط

Wait

ولا تضغط

Force Quit

غالبًا بعد 10 إلى 30 ثانية سيكمل

ماذا سيحدث بعد ذلك

سيظهر في التيرمينال شيء مثل

Calibration complete

وفي نافذة المعايرة سيصبح زر

SAVE

مفعّل

اضغط بالترتيب

SAVE

COMMIT

ماذا سيحفظ النظام

يحتوي calibration سيحفظ ملف

f_x

f_y

c_x

c_y

k_1 k_2 k_3

هذه القيم هي التي تسمح لنا بتحويل

pixel → 3D point

هذه أهم خطوة في مشروعك

لأن المرحلة القادمة ستكون

تحديد زر الأسانسير في الصورة ثم حساب موقعه الحقيقي أمام الروبوت

أي أننا سنحول

في الصورة (u , v)

إلى

في الفراغ XYZ

وهذا ما يسمح للفراع أن يضغط الزر

قبل أن نكمل

عندما ينتهي البرنامج أخبرني

هل ظهر لك

SAVE

COMMIT

أم لا

بعدما سأريك خطوة مهمة جدًا اسمها

RGB-Depth alignment test

وهي التي تؤكد أن العمق والصورة متطابقين قبل أن نبدأ تحديد الأزرار

**** Calibrating **** mono pinhole calibration... *** Added sample 72, p_x =

0.394, p_y = 0.453, p_size = 0.145, skew = 0.218 *** Added sample 73, p_x =

0.387, p_y = 0.461, p_size = 0.128, skew = 0.430 *** Added sample 74, p_x =

0.385, p_y = 0.473, p_size = 0.114, skew = 0.675 D =

[0.10580613212313277, 0.11314314860303064, -0.004149764626017555,

0.027141766973237204, 0.0] K = [673.9017858469758, 0.0,

367.288568092146, 0.0, 673.7243468622482, 258.1517978790457, 0.0, 0.0,

1.0] R = [1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0] P = [701.8192138671875,

0.0, 375.97707680660824, 0.0, 0.0, 709.8072509765625,

255.78108273503312, 0.0, 0.0, 0.0, 1.0, 0.0] None # oST version 5.0

parameters [image] width 640 height 480 [narrow_stereo] camera matrix

673.901786 0.000000 367.288568 0.000000 673.724347 258.151798

0.000000 0.000000 1.000000 distortion 0.105806 0.113143 -0.004150

0.027142 0.000000 rectification 1.000000 0.000000 0.000000 0.000000

1.000000 0.000000 0.000000 0.000000 1.000000 projection 701.819214

0.000000 375.977077 0.000000 0.000000 709.807251 255.781083 0.000000

0.000000 0.000000 1.000000 0.000000

ممتاز جدًا يا عبدالله

خلينا نفسرها (Camera Intrinsics) المعايير نجحت 100% والبيانات اللي ظهرت هي معاملات الكاميرا

بسرعة وبطريقة مفيدة لمشروعك

نتيجة المعايير

:القيم الأساسية التي حصلنا عليها

(Camera Matrix) مصفوفة الكاميرا

$f_x = 673.90$

$f_y = 673.72$

$c_x = 367.29$

$c_y = 258.15$

:المصفوفة

$K=673.90000673.720367.29258.151$

نمعناها

- **fx, fy** → (focal length) البعد البؤري بالكاميرا (بالبكسل)
- **cx, cy** → (principal point) مركز الصورة

(Distortion) معاملات التشوه

$$k1 = 0.1058$$

$$k2 = 0.1131$$

$$p1 = -0.0041$$

$$p2 = 0.0271$$

$$k3 = 0$$

هذه تستخدم لإزالة التشوه من الصورة

ماذا سنفعل بهذه القيم؟

هذه القيم هي التي تسمح لنا بتحويل

Pixel (u , v)

+

Depth Z

إلى

3D Point (X , Y , Z)

باستخدام المعادلات

$$X = (u - cx) * Z / fx$$

$$Y = (v - cy) * Z / fy$$

$$Z = \text{depth}$$

وهذا هو أساس كل مهام مشروعك

● تحديد زر الأسانسير

● تحديد مقبض الباب

● تحديد إطار باب الأسانسير

مثال بسيط

لو زر الأسانسير ظهر في الصورة عند

$$u = 400$$

$$v = 250$$

والعمق

$$Z = 1.2 \text{ m}$$

نحسب

$$X = (400 - 367.29) * 1.2 / 673.9$$

$$Y = (250 - 258.15) * 1.2 / 673.7$$

فإنحصل على موقع الزر الحقيقي بالنسبة للكاميرا

الخطوة القادمة في المشروع

الآن انتهينا من

Phase 1: Camera Calibration

المرحلة القادمة

Phase 2: RGB + Depth Alignment Test

نريد التأكد أن

- البكسل في الصورة
- يقابله نفس المكان في العمق

لأن بعض الكاميرات تكون فيها إزاحة

الاختبار الذي سنعمله

سنشغل

image + point cloud

نضع جسم صغير أمام الكاميرا ونرى هل RViz في

- النقطة في الصورة
- PointCloud تظهر في نفس المكان في الـ

بعد ذلك مباشرة

سنبدأ المرحلة الأهم في مشروعك

Phase 3: Button Detection

سنقوم بعمل

Elevator Button Detection

نستخدم

● OpenCV

● YOLO أو

ثم نحسب موقعه ثلاثي الأبعاد

قبل أن نكمل

أريد أن أعرف شيئاً مهماً

هل ظهر عندك زر

SAVE

COMMIT

وضغطتهما أم لا؟

ROS. لأن هذا يحفظ المعايير داخل نظام

ظهرو بس لسه مضغطتش

o

ممتاز يا عبدالله

h

a

p

r

t

s

d

الخطوة الآن

في نافذة المعايير اضغط بالترتيب

أولاً

SAVE

هذا سيحفظ المعاملات التي حسبها البرنامج

● camera matrix

● distortion

● projection

ثانيًا

الضغط

COMMIT

هذه الخطوة مهمة جدًا لأنها

● ترسل القيم إلى

/camera/color/camera_info

(undistorted) باستخدام الكاميرا بعد التصحيح ROS ويبدأ

بعد الضغط

سيظهر في التيرمينال شيء مثل

Wrote calibration data to ...

أو

Set camera_info

بعد ذلك

يمكنك إغلاق برنامج المعايرة

ما الذي فعلناه الآن فعليًا

الكاميرا intrinsic parameters نحن قمنا بحساب

وهذا يسمح لنا في الخطوة القادمة أن نحسب

pixel → 3D position

وهذا هو الأساس الذي سيجعل الروبوت

- يعرف مكان زر الأسانسير
 - يعرف مكان المقبض
 - يعرف مكان باب الأسانسير
- هدقة في الفراغ

المرحلة القادمة

نحن نظام الفيجن Phase 2 الآن سنبدأ

RGB-Depth Alignment Test

سنفحص هل

RGB pixel

=

Depth pixel

متطابقين

لأن هذا مهم قبل ما نبدأ تحديد الأزرار

قبل أن نكمل

نعد أن تضغط

SAVE

COMMIT

أخبرني فقط

هل ظهرت رسالة نجاح في التيرمينال؟

وسأعطيك بعدها مباشرة اختبار احترافي للتأكد أن الكاميرا جاهزة لمهام المشروع

Y
o
u

s
a
i
d
:

عايز حل للظهور المتكرر للرسله ده من ساعة مضغطت كومت

COMMIT تمام يا عبدالله، اللي حصل معك طبيعي جدًا بعد الضغط على

والرسالة التي تظهر ليست خطأ في المعايرة نفسها

خلينا نفهم السبب ثم الحل

ماذا يحدث الآن

بعد الضغط على

COMMIT

برنامج المعايرة يقوم بإرسال القيم إلى

/camera/color/set_camera_info

أي أنه يحدث إعدادات الكاميرا

بذلك يظهر **OpenCV** أثناء ذلك يحصل أحيانًا تجمّد مؤقت في واجهة

python3 is not responding

لكن الحسابات انتهت بالفعل لأنك ترى في التيرمينال

camera matrix

distortion

rectification

projection

وهذا يعني أن المعايرة تمت بنجاح

الحل الصحيح الآن

في النافذة التي ظهرت لديك

اضغط

Force Quit

لا تقلق

لن تفقد المعايرة لأن

القيم تم حفظها بالفعل

لماذا هذا آمن

بسبب **commit** معروف أن واجهته تتجمد بعد ROS2 في cameracalibrator برنامج

OpenCV GUI thread bug

وهذا طبيعي ويحدث كثيرًا

بعد إغلاق البرنامج

أغلق كل النوافذ

ثم أعد تشغيل الكاميرا فقط

```
source /opt/ros/humble/setup.bash
```

```
source ~/ros2_ws/install/setup.bash
```

```
ros2 launch astra_camera astra_pro.launch.xml
```

اختبار سريع للتأكد أن الكاميرا تعمل

تشغل

```
ros2 topic echo /camera/color/camera_info
```

يجب أن ترى القيم التي حصلنا عليها

673.9
673.7
367.2
258.1

ماذا يعني هذا للمشروع

هذا يعني أن

Phase 1 انتهت بنجاح

فهي

Camera Intrinsic Calibration

المرحلة التالية في المشروع

الخطوة القادمة التي سنبدأ بها هي

RGB + Depth alignment test

ثم بعدها مباشرة

3D point extraction

فهذا ما يمكننا من

● تحديد زر الأسانسير

● تحديد مقبض الباب

● تحديد إطار الباب

بدقة في الفراغ

سؤال سريع قبل أن نكمل

نفذ هذا الأمر وأرسل النتيجة

ros2 topic echo /camera/color/camera_info

يستخدم المعاملات الجديدة ROS حتى أتأكد أن

Y 0.0 - 0.0 - 0.0 - 1.0 - 0.0 - 0.0 - 0.0 - 1.0 p: - 570.3422047415297 - 0.0 - 319.5

u - 0.0 - 0.0 - 570.3422047415297 - 239.5 - 0.0 - 0.0 - 0.0 - 1.0 - 0.0 binning_x: 0

s
a
i
d
:

binning_y: 0 roi: x_offset: 0 y_offset: 0 height: 0 width: 0 do_rectify: false ---

header: stamp: sec: 1772799155 nanosec: 41110177 frame_id:

camera_color_optical_frame height: 480 width: 640 distortion_model:

plumb_bob d: - 0.0 - 0.0 - 0.0 - 0.0 - 0.0 k: - 570.3422047415297 - 0.0 - 319.5 -

0.0 - 570.3422047415297 - 239.5 - 0.0 - 0.0 - 1.0 r: - 1.0 - 0.0 - 0.0 - 0.0 - 1.0 -

0.0 - 0.0 - 0.0 - 1.0 p: - 570.3422047415297 - 0.0 - 319.5 - 0.0 - 0.0 -

570.3422047415297 - 239.5 - 0.0 - 0.0 - 0.0 - 1.0 - 0.0 binning_x: 0 binning_y:

0 roi: x_offset: 0 y_offset: 0 height: 0 width: 0 do_rectify: false --- header:

stamp: sec: 1772799155 nanosec: 77103087 frame_id:

camera_color_optical_frame height: 480 width: 640 distortion_model:

plumb_bob d: - 0.0 - 0.0 - 0.0 - 0.0 - 0.0 k: - 570.3422047415297 - 0.0 - 319.5 -

0.0 - 570.3422047415297 - 239.5 - 0.0 - 0.0 - 1.0 r: - 1.0 - 0.0 - 0.0 - 0.0 - 1.0 -

0.0 - 0.0 - 0.0 - 1.0 p: - 570.3422047415297 - 0.0 - 319.5 - 0.0 - 0.0 -

570.3422047415297 - 239.5 - 0.0 - 0.0 - 0.0 - 1.0 - 0.0 binning_x: 0 binning_y:

0 roi: x_offset: 0 y_offset: 0 height: 0 width: 0 do_rectify: false --- header:

stamp: sec: 1772799155 nanosec: 109121858 frame_id:

camera_color_optical_frame height: 480 width: 640 distortion_model:

plumb_bob d: - 0.0 - 0.0 - 0.0 - 0.0 - 0.0 k: - 570.3422047415297 - 0.0 - 319.5 -

0.0 - 570.3422047415297 - 239.5 - 0.0 - 0.0 - 1.0 r: - 1.0 - 0.0 - 0.0 - 0.0 - 1.0 -

0.0 - 0.0 - 0.0 - 1.0 p: - 570.3422047415297 - 0.0 - 319.5 - 0.0 - 0.0 -

570.3422047415297 - 239.5 - 0.0 - 0.0 - 0.0 - 1.0 - 0.0 binning_x: 0 binning_y:

0 roi: x_offset: 0 y_offset: 0 height: 0 width: 0 do_rectify: false --- header:

stamp: sec: 1772799155 nanosec: 141114279 frame_id:

camera_color_optical_frame height: 480 width: 640 distortion_model:

plumb_bob d: - 0.0 - 0.0 - 0.0 - 0.0 - 0.0 k: - 570.3422047415297 - 0.0 - 319.5 -

0.0 - 570.3422047415297 - 239.5 - 0.0 - 0.0 - 1.0 r: - 1.0 - 0.0 - 0.0 - 0.0 - 1.0 -

0.0 - 0.0 - 0.0 - 1.0 p: - 570.3422047415297 - 0.0 - 319.5 - 0.0 - 0.0 -

570.3422047415297 - 239.5 - 0.0 - 0.0 - 0.0 - 1.0 - 0.0 binning_x: 0 binning_y:

0 roi: x_offset: 0 y_offset: 0 height: 0 width: 0 do_rectify: false --- header:

stamp: sec: 1772799155 nanosec: 177112189 frame_id:

camera_color_optical_frame height: 480 width: 640 distortion_model:

plumb_bob d: - 0.0 - 0.0 - 0.0 - 0.0 - 0.0 k: - 570.3422047415297 - 0.0 - 319.5 -

0.0 - 570.3422047415297 - 239.5 - 0.0 - 0.0 - 1.0 r: - 1.0 - 0.0 - 0.0 - 0.0 - 1.0 -

0.0 - 0.0 - 0.0 - 1.0 p: - 570.3422047415297 - 0.0 - 319.5 - 0.0 - 0.0 -

570.3422047415297 - 239.5 - 0.0 - 0.0 - 0.0 - 1.0 - 0.0 binning_x: 0 binning_y:

0 roi: x_offset: 0 y_offset: 0 height: 0 width: 0 do_rectify: false --- header:

stamp: sec: 1772799155 nanosec: 209130790 frame_id:

camera_color_optical_frame height: 480 width: 640 distortion_model:

plumb_bob d: - 0.0 - 0.0 - 0.0 - 0.0 - 0.0 k: - 570.3422047415297 - 0.0 - 319.5 -

0.0 - 570.3422047415297 - 239.5 - 0.0 - 0.0 - 1.0 r: - 1.0 - 0.0 - 0.0 - 0.0 - 1.0 -

0.0 - 0.0 - 0.0 - 1.0 p: - 570.3422047415297 - 0.0 - 319.5 - 0.0 - 0.0 -

570.3422047415297 - 239.5 - 0.0 - 0.0 - 0.0 - 1.0 - 0.0 binning_x: 0 binning_y:

0 roi: x_offset: 0 y_offset: 0 height: 0 width: 0 do_rectify: false --- header:

stamp: sec: 1772799155 nanosec: 241110818 frame_id:

camera_color_optical_frame height: 480 width: 640 distortion_model:

plumb_bob d: - 0.0 - 0.0 - 0.0 - 0.0 - 0.0 k: - 570.3422047415297 - 0.0 - 319.5 -

0.0 - 570.3422047415297 - 239.5 - 0.0 - 0.0 - 1.0 r: - 1.0 - 0.0 - 0.0 - 0.0 - 1.0 -

0.0 - 0.0 - 0.0 - 1.0 p: - 570.3422047415297 - 0.0 - 319.5 - 0.0 - 0.0 -

570.3422047415297 - 239.5 - 0.0 - 0.0 - 0.0 - 1.0 - 0.0 binning_x: 0 binning_y:

0 roi: x_offset: 0 y_offset: 0 height: 0 width: 0 do_rectify: false --- header:

stamp: sec: 1772799155 nanosec: 277120690 frame_id:

camera_color_optical_frame height: 480 width: 640 distortion_model:

plumb_bob d: - 0.0 - 0.0 - 0.0 - 0.0 - 0.0 k: - 570.3422047415297 - 0.0 - 319.5 -

0.0 - 570.3422047415297 - 239.5 - 0.0 - 0.0 - 1.0 r: - 1.0 - 0.0 - 0.0 - 0.0 - 1.0 -

0.0 - 0.0 - 0.0 - 1.0 p: - 570.3422047415297 - 0.0 - 319.5 - 0.0 - 0.0 -

570.3422047415297 - 239.5 - 0.0 - 0.0 - 0.0 - 1.0 - 0.0 binning_x: 0 binning_y:

0 roi: x_offset: 0 y_offset: 0 height: 0 width: 0 do_rectify: false ---

ممتاز يا عبدالله

ROS. النتيجة اللي أرسلتها مهمة جدًا لأنها تكشف لنا حالة المعايرة فعليًا داخل

خلينا نحللها بهدوء كمهندس روبوتات

/camera/color/camera_info : ما الذي نراه في

:القيم الموجودة الآن

$fx = 570.34$

$fy = 570.34$

$cx = 319.5$

$cy = 239.5$

ومعاملات التشوه

$d = [0,0,0,0,0]$

ماذا يعني هذا؟

هذه ليست القيم التي حسبناها أثناء المعايرة

:القيم التي حسبناها كانت

$fx \approx 673$

$fy \approx 673$

$cx \approx 367$

$cy \approx 258$

$\text{distortion} \neq 0$

:الآن يستخدم ROS لكن

$fx = 570$

$fy = 570$

$\text{distortion} = 0$

وهذا يعني أن

الافتراضي camera_info بعد تحميل astra_camera driver

بعد إعادة تشغيل الكاميرا

هذا طبيعي مع كثير من رايفرات الكاميرات

هل هذا مشكلة؟

لا

لكن إذا أردنا دقة أعلى في الرؤية (وخصوصًا لمشروعك)

● تحديد زر الأسانسير

● تحديد المقبض

● حساب إحداثيات ثلاثية الأبعاد

يفضل أن نستخدم المعاملات التي حسبناها

لكن يمكننا الاستمرار الآن لأن الفرق ليس كارثيًا

أهم شيء تحقق الآن

هذه الرسالة تعني

camera_info يتم نشره

الكاميرا يعمل بشكل كامل pipeline أي أن

المرحلة التالية في المشروع

الآن ننتقل إلى المرحلة الثانية

Phase 2 — RGB / Depth Alignment

نريد التأكد أن

pixel في RGB

=

pixel في depth

متطابقين

لأننا لاحقًا سنستخدم

RGB → detect button

Depth → calculate distance

الاختبار الذي سنعمله الآن

افتح RViz.

ثم أضف

Image

/camera/color/image_raw

PointCloud2

/camera/depth/points

فاجعل

Fixed Frame = camera_link

ثم ضع جسمًا صغيرًا أمام الكاميرا مثل

● قلم

● إصبعك

● زر

فانظر

تظهر في نفس مكان الجسم في الصورة PointCloud هل النقاط في الـ

ماذا سيخبرنا هذا الاختبار؟

إذا كان التطابق صحيحًا

تستطيع الانتقال مباشرة إلى

Phase 3 — Button Detection

وهي الخطوة التي سنبدأ فيها

Elevator Button Detection

YOLO. أو OpenCV باستخدام

سؤال سريع قبل أن نكمل

الآن RViz في

يظهر فوق الأجسام بشكل صحيح أم أنه مزاح قليلاً عن الصورة؟ **PointCloud** مل

طيب زاي احفظ قيم المعايير بحيث كل مره اشغل الكاميرا تستخدمها

سؤال ممتاز يا عبدالله

هو حفظ ROS2 دي خطوة مهمة جداً لأن رايفر الكاميرا غالباً يرجع للإعدادات الافتراضية كل مرة. الحل الصحيح في

ملف المعايرة واستخدامه عند تشغيل الكاميرا

خلينا نمشيها خطوة خطوة بشكل نظيف

حفظ ملف المعايرة

أولاً أنشئ مجلد للمعايرة

```
mkdir -p ~/camera_calibration
```

ثم أنشئ الملف

```
gedit ~/camera_calibration/astra_rgb.yaml
```

وضع فيه القيم التي حسبناها أثناء المعايرة (القيم الصحيحة)

مثال

```
image_width: 640
```

```
image_height: 480
```

```
camera_name: astra_rgb
```

```
camera_matrix:
```

```
  rows: 3
```

```
  cols: 3
```

```
  data: [673.901786, 0.0, 367.288568,
```

```
         0.0, 673.724347, 258.151798,
```

```
         0.0, 0.0, 1.0]
```

```
distortion_model: plumb_bob
```

```
distortion_coefficients:
```

rows: 1
cols: 5
data: [0.105806, 0.113143, -0.004150, 0.027142, 0.0]
rectification_matrix:
rows: 3
cols: 3
data: [1.0,0.0,0.0,
0.0,1.0,0.0,
0.0,0.0,1.0]
projection_matrix:
rows: 3
cols: 4
data: [701.819214,0.0,375.977077,0.0,
0.0,709.807251,255.781083,0.0,
0.0,0.0,1.0,0.0]
احفظ الملف

	الخاص بالكاميرا	launch	تعديل
نحتاج أن نجعل الكاميرا تستخدم ملف المعايرة			
launch: افتح ملف			

gedit ~/ros2_ws/src/ros2_astra_camera/astra_camera/launch/astra_pro.launch.xml

node	الخاصة بالكاميرا	ابحث عن
أضف السطر التالي داخلها		

```
<param name="camera_info_url"
value="file:///home/abdullah/camera_calibration/astra_rgb.yaml"/>
```

سيصبح مثلاً

```
<node pkg="astra_camera" exec="astra_camera_node" name="astra_camera">
  <param name="camera_info_url"
value="file:///home/abdullah/camera_calibration/astra_rgb.yaml"/>
</node>
```

	إعادة بناء الباكج
launch: بعد تعديل	


```
cd ~/ros2_ws  
colcon build
```

تم

```
source install/setup.bash
```

اختبار

تشغيل الكاميرا

```
ros2 launch astra_camera astra_pro.launch.xml
```

تم افحص

```
ros2 topic echo /camera/color/camera_info
```

نحتاج أن ترى

$f_x \approx 673$

$f_y \approx 673$

$\text{distortion} \neq 0$

وليس القيم القديمة

الآن الكاميرا جاهزة للمشروع

نعد حفظ المعايرة أصبح لدينا

RGB calibrated ✓

Depth stream ✓

Point cloud ✓

نحن الآن جاهزون للمرحلة التالية

Phase 3 — استخراج نقطة ثلاثية الأبعاد من الصورة

هذه الخطوة التي ستسمح للروبوت أن يعرف

● مكان زر الأسانسير

● مكان المقبض

● مكان إطار الباب

نصيحة مهمة لمشروعك

سنحتاج قريباً أيضاً إلى **Depth camera**، بما أنك تستخدم

RGB ↔ Depth registration

حتى تكون النقطة في الصورة مرتبطة بالعمق

سنكتبها للمشروع وهي **Node** لو حابب، الخطوة القادمة أشرح لك أول

pixel → 3D point node

وهي أساس كل مهام الروبوت التي ذكرتها

```
<launch> <!-- unique camera name--> <arg name="camera_name"
default="camera"/> <!-- Hardware depth registration --> <arg
name="depth_registration" default="false"/> <arg name="serial_number"
default=""/> <arg name="device_num" default="1"/> <arg
name="vendor_id" default="0"/> <arg name="product_id" default="0"/>
<arg name="enable_point_cloud" default="true"/> <arg
name="enable_colored_point_cloud" default="false"/> <arg
name="point_cloud_qos" default="default"/> <arg name="connection_delay"
default="100"/> <arg name="color_width" default="640"/> <arg
name="color_height" default="480"/> <arg name="color_fps" default="30"/>
<arg name="enable_color" default="true"/> <arg name="flip_color"
default="false"/> <arg name="color_qos" default="default"/> <arg
name="color_camera_info_qos" default="default"/> <arg
name="depth_width" default="640"/> <arg name="depth_height"
default="480"/> <arg name="depth_fps" default="30"/> <arg
name="enable_depth" default="true"/> <arg name="flip_depth"
default="false"/> <arg name="depth_qos" default="default"/> <arg
name="depth_camera_info_qos" default="default"/> <arg name="ir_width"
default="640"/> <arg name="ir_height" default="480"/> <arg name="ir_fps"
default="30"/> <arg name="enable_ir" default="true"/> <arg name="flip_ir"
```

```

default="false"/> <arg name="ir_qos" default="default"/> <arg
name="ir_camera_info_qos" default="default"/> <arg name="publish_tf"
default="true"/> <arg name="tf_publish_rate" default="10.0"/> <arg
name="ir_info_url" default=""/> <arg name="color_info_url" default=""/>
<arg name="color_roi_x" default="-1"/> <arg name="color_roi_y" default="-
1"/> <arg name="color_roi_width" default="-1"/> <arg
name="color_roi_height" default="-1"/> <arg name="depth_roi_x" default="-
1"/> <arg name="depth_roi_y" default="-1"/> <arg name="depth_roi_width"
default="-1"/> <arg name="depth_roi_height" default="-1"/> <arg
name="depth_scale" default="1"/> <arg
name="color_depth_synchronization" default="false"/> <arg
name="use_uvc_camera" default="true"/> <arg name="uvc_vendor_id"
default="0x2bc5"/> <arg name="uvc_product_id" default="0x0501"/> <arg
name="uvc_retry_count" default="100"/> <arg name="uvc_camera_format"
default="mjpeg"/> <arg name="uvc_flip" default="false"/> <arg
name="oni_log_level" default="verbose"/> <arg name="oni_log_to_console"
default="false"/> <arg name="oni_log_to_file" default="false"/> <arg
name="enable_d2c_viewer" default="false"/> <arg
name="enable_publish_extrinsic" default="false"/> <group> <push-ros-
namespace namespace="$(var camera_name)"/> <node name="camera"
pkg="astra_camera" exec="astra_camera_node" output="screen"> <param
name="camera_name" value="$(var camera_name)"/> <param
name="depth_registration" value="$(var depth_registration)"/> <param
name="serial_number" value="$(var serial_number)"/> <param
name="device_num" value="$(var device_num)"/> <param
name="vendor_id" value="$(var vendor_id)"/> <param name="product_id"
value="$(var product_id)"/> <param name="enable_point_cloud" value="$
(var enable_point_cloud)"/> <param name="enable_colored_point_cloud"
value="$(var enable_colored_point_cloud)"/> <param

```

```
name="point_cloud_qos" value="$ (var point_cloud_qos)"/> <param
name="connection_delay" value="$ (var connection_delay)"/> <param
name="color_width" value="$ (var color_width)"/> <param
name="color_height" value="$ (var color_height)"/> <param
name="color_fps" value="$ (var color_fps)"/> <param name="enable_color"
value="$ (var enable_color)"/> <param name="flip_color" value="$ (var
flip_color)"/> <param name="color_qos" value="$ (var color_qos)"/> <param
name="color_camera_info_qos" value="$ (var color_camera_info_qos)"/>
<param name="depth_width" value="$ (var depth_width)"/> <param
name="depth_height" value="$ (var depth_height)"/> <param
name="depth_fps" value="$ (var depth_fps)"/> <param name="flip_depth"
value="$ (var flip_depth)"/> <param name="enable_depth" value="$ (var
enable_depth)"/> <param name="depth_qos" value="$ (var depth_qos)"/>
<param name="depth_camera_info_qos" value="$ (var
depth_camera_info_qos)"/> <param name="ir_width" value="$ (var
ir_width)"/> <param name="ir_height" value="$ (var ir_height)"/> <param
name="ir_fps" value="$ (var ir_fps)"/> <param name="enable_ir" value="$
(var enable_ir)"/> <param name="flip_ir" value="$ (var flip_ir)"/> <param
name="ir_qos" value="$ (var ir_qos)"/> <param name="ir_camera_info_qos"
value="$ (var ir_camera_info_qos)"/> <param name="publish_tf" value="$ (var
publish_tf)"/> <param name="tf_publish_rate" value="$ (var
tf_publish_rate)"/> <param name="ir_info_url" value="$ (var ir_info_url)"/>
<param name="color_info_url" value="$ (var color_info_url)"/> <param
name="color_roi_x" value="$ (var color_roi_x)"/> <param name="color_roi_y"
value="$ (var color_roi_y)"/> <param name="color_roi_width" value="$ (var
color_roi_width)"/> <param name="color_roi_height" value="$ (var
color_roi_height)"/> <param name="depth_roi_x" value="$ (var
depth_roi_x)"/> <param name="depth_roi_y" value="$ (var depth_roi_y)"/>
<param name="depth_roi_width" value="$ (var depth_roi_width)"/> <param
```

```

name="depth_roi_height" value="$(var depth_roi_height)"/> <param
name="depth_scale" value="$(var depth_scale)"/> <param
name="color_depth_synchronization" value="$(var
color_depth_synchronization)"/> <param name="use_uvc_camera" value="$(
(var use_uvc_camera)"/> <param name="uvc_vendor_id" value="$(var
uvc_vendor_id)"/> <param name="uvc_product_id" value="$(var
uvc_product_id)"/> <param name="uvc_retry_count" value="$(var
uvc_retry_count)"/> <param name="uvc_camera_format" value="$(var
uvc_camera_format)"/> <param name="uvc_flip" value="$(var uvc_flip)"/>
<param name="oni_log_level" value="$(var oni_log_level)"/> <param
name="oni_log_to_console" value="$(var oni_log_to_console)"/> <param
name="oni_log_to_file" value="$(var oni_log_to_file)"/> <param
name="enable_d2c_viewer" value="$(var enable_d2c_viewer)"/> <param
name="enable_publish_extrinsic" value="$(var enable_publish_extrinsic)"/>
<remap from="/$(var camera_name)/depth/color/points" to="/$(var
camera_name)/depth_registered/points"/> </node> </group> </launch>

```

ملف الاقلاع الخاص بالكاميرا عدل عليه

ممتاز يا عبدالله

اللي أرسلته **جيد جدًا** وفيه بالفعل المكان الصحيح لإضافة ملف المعايرة. الجميل أن الرايفر نفسه **launch file**

نيوفر برامتر جاهز اسمه

color_info_url

calibration file. وهذا هو المكان الصحيح لتحميل

إذن لن نعدل كثيرًا في الملف، فقط سنربطه بملف المعايرة الذي أنشأناه

أولًا تأكد أن ملف المعايرة موجود

:المفروض أن الملف هنا

/home/abdullah/camera_calibration/astra_rgb.yaml

لو لم تنشئه بعد

```
mkdir -p ~/camera_calibration
```

```
gedit ~/camera_calibration/astra_rgb.yaml
```

لم ضع فيه القيم التي حصلنا عليها من المعايرة

launch file تعديل

:ابحث في الملف عن هذا السطر

```
<arg name="color_info_url" default=""/>
```

:غيره إلى

```
<arg name="color_info_url"
```

```
default="file:///home/abdullah/camera_calibration/astra_rgb.yaml"/>
```

لاحظ

```
file:///
```

لازم تكون موجودة

لا تحتاج تعديل أي شيء آخر

node: لأنك بالفعل عندك في

```
<param name="color_info_url" value="$(var color_info_url)"/>
```

سيأخذ القيمة التي وضعناها launch وهذا يعني أن

تشغيل الكاميرا

:بعد التعديل شغل الكاميرا

```
source /opt/ros/humble/setup.bash
```

```
source ~/ros2_ws/install/setup.bash
```

```
ros2 launch astra_camera astra_pro.launch.xml
```

اختبار أن المعايرة تُستخدم

تشغل

ros2 topic echo /camera/color/camera_info

يجب أن ترى

$f_x \approx 673$

$f_y \approx 673$

distortion $\neq 0$

تُؤليص القيم القديمة

570

distortion = 0

لماذا هذا مهم جدًا لمشروعك

هذه القيم ستستخدم لاحقًا لحساب

pixel $\rightarrow$ 3D coordinate

وهذا أساس مهام المشروع

● تحديد زر الأسانسير

● تحديد المقبض

● تحديد إطار باب الأسانسير

المرحلة القادمة في مشروعك

بما أن

☐ الكاميرا تعمل

☒ العمق يعمل

☒ المعايرة جاهزة

سنبدأ المرحلة التالية

Phase 2 — استخراج نقطة ثلاثية الأبعاد من الصورة

حقيقية في المشروع Vision وهذه هي أول خوارزمية

لأي نقطة **D** يوضح كيف نحصل على إحداثيات **3 RViz** لو حابب، في الخطوة القادمة سأريك اختبار احترافي في

تضغط عليها بالماوس

وهذا سيكون أول خطوة فعلية نحو تحديد زر الأسانسير

طيب عدل وارسل التعديل كامل لاستبدله بالحالي

تمام يا عبدالله

بحيث يستخدم ملف المعايرة تلقائيًا كل مرة تشغل الكاميرا **launch** أنا عدلت ملف الي

:التعديل الوحيد الحقيقي هو ربط

color_info_url

بملف المعايرة

ضع ملف المعايرة هنا

/home/abdullah/camera_calibration/astra_rgb.yaml

بالكامل بالكود التالي **launch** ثم استبدل ملف الـ

المعدل بالكامل launch file

<launch>

<!-- unique camera name-->

<arg name="camera_name" default="camera"/>

<!-- Hardware depth registration -->

<arg name="depth_registration" default="false"/>

<arg name="serial_number" default=""/>

<arg name="device_num" default="1"/>

<arg name="vendor_id" default="0"/>

<arg name="product_id" default="0"/>

<arg name="enable_point_cloud" default="true"/>

<arg name="enable_colored_point_cloud" default="false"/>

<arg name="point_cloud_qos" default="default"/>

<arg name="connection_delay" default="100"/>

<arg name="color_width" default="640"/>

<arg name="color_height" default="480"/>

<arg name="color_fps" default="30"/>

<arg name="enable_color" default="true"/>

<arg name="flip_color" default="false"/>

<arg name="color_qos" default="default"/>


```

<arg name="color_camera_info_qos" default="default"/>
<arg name="depth_width" default="640"/>
<arg name="depth_height" default="480"/>
<arg name="depth_fps" default="30"/>
<arg name="enable_depth" default="true"/>
<arg name="flip_depth" default="false"/>
<arg name="depth_qos" default="default"/>
<arg name="depth_camera_info_qos" default="default"/>
<arg name="ir_width" default="640"/>
<arg name="ir_height" default="480"/>
<arg name="ir_fps" default="30"/>
<arg name="enable_ir" default="true"/>
<arg name="flip_ir" default="false"/>
<arg name="ir_qos" default="default"/>
<arg name="ir_camera_info_qos" default="default"/>
<arg name="publish_tf" default="true"/>
<arg name="tf_publish_rate" default="10.0"/>
<!-- calibration files -->
<arg name="ir_info_url" default=""/>
<arg name="color_info_url"
    default="file:///home/abdullah/camera_calibration/astra_rgb.yaml"/>
<arg name="color_roi_x" default="-1"/>
<arg name="color_roi_y" default="-1"/>
<arg name="color_roi_width" default="-1"/>
<arg name="color_roi_height" default="-1"/>
<arg name="depth_roi_x" default="-1"/>
<arg name="depth_roi_y" default="-1"/>
<arg name="depth_roi_width" default="-1"/>
<arg name="depth_roi_height" default="-1"/>
<arg name="depth_scale" default="1"/>
<arg name="color_depth_synchronization" default="false"/>
<arg name="use_uvc_camera" default="true"/>
<arg name="uvc_vendor_id" default="0x2bc5"/>
<arg name="uvc_product_id" default="0x0501"/>
<arg name="uvc_retry_count" default="100"/>
<arg name="uvc_camera_format" default="mjpeg"/>
<arg name="uvc_flip" default="false"/>
<arg name="oni_log_level" default="verbose"/>
<arg name="oni_log_to_console" default="false"/>
<arg name="oni_log_to_file" default="false"/>
<arg name="enable_d2c_viewer" default="false"/>
<arg name="enable_publish_extrinsic" default="false"/>
<group>

```

```

<push-ros-namespace namespace="$(var camera_name)"/>
<node name="camera"
      pkg="astra_camera"
      exec="astra_camera_node"
      output="screen">
  <param name="camera_name" value="$(var camera_name)"/>
  <param name="depth_registration" value="$(var
depth_registration)"/>
  <param name="serial_number" value="$(var serial_number)"/>
  <param name="device_num" value="$(var device_num)"/>
  <param name="vendor_id" value="$(var vendor_id)"/>
  <param name="product_id" value="$(var product_id)"/>
  <param name="enable_point_cloud" value="$(var
enable_point_cloud)"/>
  <param name="enable_colored_point_cloud" value="$(var
enable_colored_point_cloud)"/>
  <param name="point_cloud_qos" value="$(var
point_cloud_qos)"/>
  <param name="connection_delay" value="$(var
connection_delay)"/>
  <param name="color_width" value="$(var color_width)"/>
  <param name="color_height" value="$(var color_height)"/>
  <param name="color_fps" value="$(var color_fps)"/>
  <param name="enable_color" value="$(var enable_color)"/>
  <param name="flip_color" value="$(var flip_color)"/>
  <param name="color_qos" value="$(var color_qos)"/>
  <param name="color_camera_info_qos" value="$(var
color_camera_info_qos)"/>
  <!-- calibration file -->
  <param name="color_info_url" value="$(var color_info_url)"/>
  <param name="depth_width" value="$(var depth_width)"/>
  <param name="depth_height" value="$(var depth_height)"/>
  <param name="depth_fps" value="$(var depth_fps)"/>
  <param name="enable_depth" value="$(var enable_depth)"/>
  <param name="flip_depth" value="$(var flip_depth)"/>
  <param name="depth_qos" value="$(var depth_qos)"/>
  <param name="depth_camera_info_qos" value="$(var
depth_camera_info_qos)"/>
  <param name="ir_width" value="$(var ir_width)"/>
  <param name="ir_height" value="$(var ir_height)"/>
  <param name="ir_fps" value="$(var ir_fps)"/>
  <param name="enable_ir" value="$(var enable_ir)"/>
  <param name="flip_ir" value="$(var flip_ir)"/>

```

```

    <param name="ir_qos" value="$(var ir_qos)"/>
    <param name="ir_camera_info_qos" value="$(var
ir_camera_info_qos)"/>
    <param name="publish_tf" value="$(var publish_tf)"/>
    <param name="tf_publish_rate" value="$(var tf_publish_rate)"/>
    <param name="ir_info_url" value="$(var ir_info_url)"/>
    <param name="color_roi_x" value="$(var color_roi_x)"/>
    <param name="color_roi_y" value="$(var color_roi_y)"/>
    <param name="color_roi_width" value="$(var color_roi_width)"/>
    <param name="color_roi_height" value="$(var
color_roi_height)"/>
    <param name="depth_roi_x" value="$(var depth_roi_x)"/>
    <param name="depth_roi_y" value="$(var depth_roi_y)"/>
    <param name="depth_roi_width" value="$(var
depth_roi_width)"/>
    <param name="depth_roi_height" value="$(var
depth_roi_height)"/>
    <param name="depth_scale" value="$(var depth_scale)"/>
    <param name="color_depth_synchronization" value="$(var
color_depth_synchronization)"/>
    <param name="use_uvc_camera" value="$(var
use_uvc_camera)"/>
    <param name="uvc_vendor_id" value="$(var uvc_vendor_id)"/>
    <param name="uvc_product_id" value="$(var uvc_product_id)"/>
    <param name="uvc_retry_count" value="$(var
uvc_retry_count)"/>
    <param name="uvc_camera_format" value="$(var
uvc_camera_format)"/>
    <param name="uvc_flip" value="$(var uvc_flip)"/>
    <param name="oni_log_level" value="$(var oni_log_level)"/>
    <param name="oni_log_to_console" value="$(var
oni_log_to_console)"/>
    <param name="oni_log_to_file" value="$(var oni_log_to_file)"/>
    <param name="enable_d2c_viewer" value="$(var
enable_d2c_viewer)"/>
    <param name="enable_publish_extrinsic" value="$(var
enable_publish_extrinsic)"/>
    <remap from="/$(var camera_name)/depth/color/points"
to="/$(var camera_name)/depth_registered/points"/>
  </node>
</group>
</launch>

```

بعد استبدال الملف

تشغل

```
cd ~/ros2_ws
```

```
colcon build
```

```
source install/setup.bash
```

تم

```
ros2 launch astra_camera astra_pro.launch.xml
```

تم تحقق

```
ros2 topic echo /camera/color/camera_info
```

يجب أن ترى

```
fx ≈ 673
```

```
distortion ≠ 0
```

خطوة مهمة لمشروعك

بمجرد أن نتأكد أن المعايير تُحمَل تلقائياً، سنبدأ مباشرة المرحلة التالية:

تحويل أي نقطة تضغط عليها في الصورة إلى إحداثيات ثلاثية الأبعاد أمام الروبوت

وهذه الخطوة هي أساس

● ضغط زر الأسانسير

● فتح المقبض

● تحديد باب الأسانسير

مباشرة pixel لأي D point لاستخراج 3 RViz وسأريك بعدها طريقة احترافية في

Y
o
u

s
a
i
d
:

تمام يا عبدالله، الصورة وضحت مشكلتين مختلفتين
خلينا نفصلهم ونحلهم بسرعة

المشكلة الأولى

ROS لا يجد الحزمة **astra_camera**

الرسالة:

Package 'astra_camera' not found
searching: ['/opt/ros/humble']

نرى فقط ROS هذا يعني أن

/opt/ros/humble

الخاص بك workspace ولا يرى

~/ros2_ws

الحل

قبل أي تشغيل لازم تعمل

source /opt/ros/humble/setup.bash

source ~/ros2_ws/install/setup.bash

ثم شغل

ros2 launch astra_camera astra_pro.launch.xml

المشكلة الثانية

الكاميرا مشغولة بالفعل

:في التيرمينال الآخر يظهر

Resource busy

Failed to open device

أخرى node هذا يعني أن الكاميرا مفتوحة بالفعل بواسطة

ROS لا يستطيع فتح نفس الكاميرا مرتين

الحل

:اقفل كل النودات القديمة

:اكتب

`kill -f astra`

:أو ببساطة أغلق كل التيرمينالات التي تشغل الكاميرا

:ثم افتح تيرمينال جديد فقط

التسلسل الصحيح لتشغيل الكاميرا

:جديد واكتب Terminal افتح

`cd ~/ros2_ws`

`source /opt/ros/humble/setup.bash`

`source install/setup.bash`

:ثم

`ros2 launch astra_camera astra_pro.launch.xml`

يرى الحزمة ROS للتأكد أن

:نفذ

ros2 pkg list | **grep** astra

يجب أن ترى

astra_camera

astra_camera_msgs

اختبار سريع

بعد تشغيل الكاميرا

ros2 topic list | **grep** camera

يجب أن ترى

/camera/color/image_raw

/camera/depth/image_raw

/camera/depth/points

نقطة مهمة لمشروعك

بعد أن نثبت أن المعايير يتم تحميلها تلقائيًا، سننتقل مباشرة إلى

استخراج إحداثيات ثلاثية الأبعاد لأي نقطة في الصورة

وهذا هو الأساس لعمل

● ضغط زر الأسانسير

● تحديد المقبض

● تحديد باب الأسانسير

سؤال سريع

نفذ هذا وأرسل الناتج

ros2 pkg list | **grep** astra

يعمل بشكل صحيح قبل أن نكمل المرحلة التالية من الفيजन حتى أتأكد أن workspace

Y
o
u

s
a
i
d
:

تمام يا عبدالله، من الصورة واضح عندنا مشكلتين واضحين وسأعطيك الحل الدقيق خطوة بخطوة

المشكلة 1 (الأهم)

الكاميرا مفتوحة أكثر من مرة

في التيرمينال يظهر

Resource busy

Failed to open device

أخرى ROS هذا يعني أن الكاميرا مشغولة بالفعل بنود

أكثر من مرةastra_camera وأنا أرى أنك شغلت

الحل الصحيح

أقتل كل العمليات التي تستخدم الكاميرا

جديد: Terminal كُتب في

ps aux | grep astra

تم تأكد

node. يجب ألا يظهر أي

المشكلة 2

ROS **astra_camera** لا يرى حزمة

الأمر الذي نفذته

```
ros2 pkg list | grep astra
```

لم يعط أي نتيجة

workspace. لا **source** لم يتم عمل **ROS** هذا يعني أن

الحل

جديد يجب تنفيذ Terminal في كل

```
cd ~/ros2_ws
```

```
source /opt/ros/humble/setup.bash
```

```
source install/setup.bash
```

ثم تحقق

```
ros2 pkg list | grep astra
```

يجب أن يظهر شيء مثل

```
astra_camera
```

```
astra_camera_msgs
```

الآن تشغيل الكاميرا بشكل صحيح

جديد فقط واكتب Terminal افتح

```
cd ~/ros2_ws
```

```
source /opt/ros/humble/setup.bash
```

```
source install/setup.bash
```

```
ros2 launch astra_camera astra_pro.launch.xml
```

للتأكد أن الكاميرا تعمل

آخر Terminal افتح

```
ros2 topic list | grep camera
```

يجب أن ترى

/camera/color/image_raw

/camera/depth/image_raw

/camera/depth/points

ملاحظة مهمة جدًا

أرى أنك تشغل

astra_camera_node

terminals. أكثر من مرة في عدة

القاعدة

واحدة فقط **Node** للكاميرا يجب أن تعمل

نصيحة مهمة لمشروعك

.bashrc إلى workspace كل مرة، أضف source حتى لا تضطر لعمل

اكتب

```
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
```

جديد وسيعمل تلقائيًا terminal بعدها افتح

قبل أن نكمل مشروع الرؤية

أريدك أن تنفذ هذا وترسل النتيجة

```
ros2 pkg list | grep astra
```

لأن إذا لم تظهر الحزمة فهذا يعني أن

لم يتم بناؤه بشكل صحيح workspace

للمشروع vision بعد حل هذه النقطة سنبدأ مباشرة الخطوة الأولى الحقيقية في

في الصورة، وهي الخطوة التي ستجعل الروبوت يعرف مكان زر pixel لأي D position استخراج 3

الأسانسير

[astra_camera_node-1] [INFO] [1772800159.383461096] [camera.camera]:
o
uvnc config: vendor_id: 2bc5 [astra_camera_node-1] product_id: 501
[astra_camera_node-1] width: 640 [astra_camera_node-1] height: 480
a
[astra_camera_node-1] fps: 30 [astra_camera_node-1] serial_number:
d
17110911331 [astra_camera_node-1] format: mjpeg [astra_camera_node-1]
frame_id: camera_color_frame [astra_camera_node-1] optical_frame_id :
camera_color_optical_frame [astra_camera_node-1] [astra_camera_node-1]
unsupported descriptor subtype VS_COLORFORMAT [astra_camera_node-1]
unsupported descriptor subtype VS_COLORFORMAT [astra_camera_node-1]
[INFO] [1772800160.374246010] [camera.camera]: open camera success
[astra_camera_node-1] [INFO] [1772800165.661827931] [camera.camera]: set
depth video mode Resolution :640x480@30Hz [astra_camera_node-1] format
PIXEL_FORMAT_DEPTH_1_MM [astra_camera_node-1] [WARN]
[1772800165.661841797] [camera.camera]: Publishing dynamic camera
transforms (/tf) at 10 Hz [astra_camera_node-1] [INFO]
[1772800165.662333986] [camera.camera]: set ir video mode
Resolution :640x480@30Hz [astra_camera_node-1] format
[astra_camera_node-1] [INFO] [1772800165.672094666] [camera.camera]:
depth is started [astra_camera_node-1] [INFO] [1772800165.677927656]
[camera.camera]: ir is started [astra_camera_node-1] [INFO]
[1772800165.677954657] [camera.camera]: Start UVC camera
[astra_camera_node-1] [INFO] [1772800165.704774736] [camera.camera]: set
uvnc mode 640x480@30 format UVC_FRAME_FORMAT_MJPEG
[astra_camera_node-1] attempt to claim already-claimed interface 1
[astra_camera_node-1] [INFO] [1772800166.769454524] [camera.camera]:
camera calibration URL:
file:///home/abdullah/camera_calibration/astra_rgb.yaml [astra_camera_node-
1] [INFO] [1772800167.541934442] [camera.camera]: device started.
[astra_camera_node-1] [WARN] [1772800168.295807576] [camera.camera]:

[rgb_camera] does not match astra_rgb in file

/home/abdullah/camera_calibration/astra_rgb.yaml

ممتاز يا عبدالله

الكاميرا الآن تعمل فعلاً والمعايرة يتم تحميلها، لكن بقي تحذير بسيط فقط. دعنا نحلله

ما الذي نجح الآن

في اللوج يظهر

camera calibration URL:

file:///home/abdullah/camera_calibration/astra_rgb.yaml

هذا يعني أن

وجد ملف المعايرة ROS

الكاميرا قرأت الملف

المعدل يعمل launch file

إذن الخطوة التي عملناها نجحت

التحذير الذي ظهر

[rgb_camera] does not match astra_rgb in file

هذا يعني

اسم الكاميرا داخل ملف المعايرة

astra_rgb

لكن الرايفر يستخدم اسم

rgb_camera

فقط عدم تطابق أسماء، وليس خطأ

الحل الصحيح

افتح ملف المعايرة

gedit ~/camera_calibration/astra_rgb.yaml

تغيير هذا السطر

camera_name: astra_rgb

إلى

camera_name: rgb_camera

ثم احفظ الملف

لماذا نفعل هذا

نرايفر الكاميرا يتحقق أن

camera_name in yaml

=

camera_name in driver

حتى يتأكد أن الملف خاص بهذه الكاميرا

بعد التعديل

rebuild. لا تحتاج

فقط أعد تشغيل الكاميرا

pkll -f astra

ثم

ros2 launch astra_camera astra_pro.launch.xml

يجب أن يخطفي التحذير

أين نحن الآن في مشروعك

حتى الآن أنجزنا

Phase 1 — Camera Setup

☐ تشغيل الكاميرا

☒ depth تشغيل

✓ تشغيل point cloud

✓ معايرة الكاميرا

✓ تحميل المعايرة تلقائيًا

وهذه خطوة كبيرة جدًا في مشروعك

المرحلة التالية في مشروعك

سنبدأ الآن

من الصورة D استخراج إحداثيات 3 — Phase 2

سنفعل التالي

pixel (u,v)

+

depth

↓

3D point (X,Y,Z)

وهذا هو الأساس لعمل

● ضغط زر الأسانسير

● فتح المقبض

● تحديد إطار الباب

الخطوة القادمة (مهمة جدًا)

تسمح لك أن RViz سأريك طريقة في

تضغط على أي نقطة في الصورة وتعرف إحداثياتها ثلاثية الأبعاد

vision manipulation robotics. وهذه أول خطوة حقيقية في

لكن قبل أن نبدأ، أريد التأكد من شيء واحد

نفذ هذا وأرسل الناتج

ros2 topic list | **grep** camera

الخاصة بالكاميرا تعمل قبل أن نبدأ استخراج النقاط ثلاثية الأبعاد topics لتتأكد أن كل الـ

ros2 launch astra_camera astra_pro.launch.xml [INFO] [launch]: All log files

can be found below /home/abdullah/.ros/log/2026-03-06-14-34-49-015791-

abdullah-sherif-10530 [INFO] [launch]: Default logging verbosity is set to INFO

[INFO] [astra_camera_node-1]: process started with pid [10562]

[astra_camera_node-1] Warning: class_loader.impl: SEVERE WARNING!!! A

namespace collision has occurred with plugin factory for class

rclcpp_components::NodeFactoryTemplate<astra_camera::OBCameraNodeFact

ory>. New factory will OVERWRITE existing one. This situation occurs when

libraries containing plugins are directly linked against an executable (the one

running right now generating this message). Please separate plugins out into

their own library or just don't link against the library and use either

class_loader::ClassLoader/MultiLibraryClassLoader to open.

[astra_camera_node-1] at line 253 in

/opt/ros/humble/include/class_loader/class_loader/class_loader_core.hpp

[astra_camera_node-1] [INFO] [1772800489.835883117] [camera.camera]: init

done. [astra_camera_node-1] [INFO] [1772800489.835922020]

[camera.camera]: Waiting for device connection... [astra_camera_node-1]

[INFO] [1772800489.836006798] [device_listener]: Found 1 devices

[astra_camera_node-1] [INFO] [1772800489.836021115] [camera.camera]:

Trying to open device: 2bc5/0403@3/5 [astra_camera_node-1] [INFO]

[1772800490.001360859] [camera.camera]: Device connected: Astra serial

number: 17110911331 [astra_camera_node-1] [INFO]

[1772800490.001419137] [camera.camera]: starting device

[astra_camera_node-1] [INFO] [1772800490.021951731] [camera.camera]: set

depth video mode Resolution :640x480@30Hz [astra_camera_node-1] format

PIXEL_FORMAT_DEPTH_1_MM [astra_camera_node-1] [INFO]

[1772800490.022262341] [camera.camera]: set ir video mode

Resolution :640x480@30Hz [astra_camera_node-1] format

[astra_camera_node-1] [INFO] [1772800490.029160733] [camera.camera]:
open uvc camera [astra_camera_node-1] [INFO] [1772800490.050171269]
[camera.camera]: uvc config: vendor_id: 2bc5 [astra_camera_node-1]
product_id: 501 [astra_camera_node-1] width: 640 [astra_camera_node-1]
height: 480 [astra_camera_node-1] fps: 30 [astra_camera_node-1]
serial_number: 17110911331 [astra_camera_node-1] format: mjpeg
[astra_camera_node-1] frame_id: camera_color_frame [astra_camera_node-1]
optical_frame_id : camera_color_optical_frame [astra_camera_node-1]
[astra_camera_node-1] unsupported descriptor subtype VS_COLORFORMAT
[astra_camera_node-1] unsupported descriptor subtype VS_COLORFORMAT
[astra_camera_node-1] [INFO] [1772800490.060947283] [camera.camera]:
open camera success [astra_camera_node-1] [INFO] [1772800490.063103600]
[camera.camera]: set depth video mode Resolution :640x480@30Hz
[astra_camera_node-1] format PIXEL_FORMAT_DEPTH_1_MM
[astra_camera_node-1] [WARN] [1772800490.063114109] [camera.camera]:
Publishing dynamic camera transforms (/tf) at 10 Hz [astra_camera_node-1]
[INFO] [1772800490.063573036] [camera.camera]: set ir video mode
Resolution :640x480@30Hz [astra_camera_node-1] format
[astra_camera_node-1] [INFO] [1772800490.073162434] [camera.camera]:
depth is started [astra_camera_node-1] [INFO] [1772800490.078786577]
[camera.camera]: ir is started [astra_camera_node-1] [INFO]
[1772800490.078813407] [camera.camera]: Start UVC camera
[astra_camera_node-1] [INFO] [1772800490.105647909] [camera.camera]: set
uvc mode 640x480@30 format UVC_FRAME_FORMAT_MJPEG
[astra_camera_node-1] attempt to claim already-claimed interface 1
[astra_camera_node-1] [INFO] [1772800490.225790539] [camera.camera]:
device started. [astra_camera_node-1] [INFO] [1772800490.471657242]

[camera.camera]: camera calibration URL:

file:///home/abdullah/camera_calibration/astra_rgb.yaml

تمام يا عبدالله

الذي أرسلته ممتاز جدًا والكاميرا تعمل الآن بشكل صحيح. دعنا نحلله بسرعة كمهندس روبوتات حتى نتأكد log

أن كل شيء جاهز للمرحلة التالية

حالة الكاميرا الآن

:السطر المهم

device started

:هذا يعني

☐ الكاميرا تم التعرف عليها

☐ depth stream يعمل

☐ RGB stream يعمل

☐ IR يعمل

تأكيد تحميل المعايرة

:السطر المهم

camera calibration URL:

file:///home/abdullah/camera_calibration/astra_rgb.yaml

:هذا يعني

☐ ملف المعايرة يتم تحميله تلقائيًا

☐ launch file الذي عدلناه يعمل

☐ الكاميرا لن تعود للقيم الافتراضية

التحذيرات الموجودة

التحذير الأول

namespace collision plugin factory

ليس له تأثير على تشغيل الكاميرا **ROS2** هذا تحذير معروف في

يمكن تجاهله

الرسالة

attempt to claim already-claimed interface

ولا تؤثر UVC هذه تظهر في كثير من كاميرات

نظالما ترى

device started

فكل شيء طبيعي

Topics الآن دعنا نتأكد من الـ

جديد واكتب Terminal افتح

ros2 topic list | **grep** camera

يجب أن ترى تقريباً

/camera/color/image_raw

/camera/color/camera_info

/camera/depth/image_raw

/camera/depth/camera_info

/camera/depth/points

/camera/ir/image_raw

الفيجن يعمل pipeline لو ظهرت هذه فكل

أين نحن الآن في المشروع

وصلنا إلى نهاية المرحلة

Phase 1 — Camera Setup

تم إنجاز

☐ تشغيل الكاميرا

☒ تشغيل depth

☒ تشغيل point cloud

☒ calibration

☒ تحميل calibration تلقائي

☒ ROS topics تعمل

هذه خطوة كبيرة جدًا

المرحلة التالية في مشروعك

حقيقية في المشروع Vision سنبدأ الآن أول خطوة

Phase 2 — استخراج نقطة ثلاثية الأبعاد

سنقوم بتحويل

pixel (u,v)

إلى

3D coordinate (X,Y,Z)

نستخدم

● depth image

● camera matrix

فهذا هو الأساس الذي سيجعل الروبوت يعرف مكان

● زر الأسانسير

● المقبض

● باب الأسانسير

قبل أن نبدأ المرحلة التالية

نفذ هذا الأمر وأرسل الناتج

```
ros2 topic list | grep camera
```

لأي نقطة تضغط عليها في **D** يسمح لك بالحصول على إحداثيات **3 RViz** بعدما سأعطيك اختبار احترافي في

الصورة

للمشروع **Vision** وهذا سيكون أول خطوة فعلية في

```
ros2 topic list | grep camera /camera/color/camera_info
```

```
/camera/color/image_raw /camera/depth/camera_info
```

```
/camera/depth/image_raw /camera/depth/points /camera/ir/camera_info
```

```
/camera/ir/image_raw abdullah@abdullah-sherif:~/ros2_ws$
```

ممتاز يا عبدالله

الكاميرا كامل ويعمل بشكل صحيح **pipeline** الناتج الذي أرسلته يعني أن

الموجودة **topics** إلى

```
/camera/color/image_raw
```

```
/camera/color/camera_info
```

```
/camera/depth/image_raw
```

```
/camera/depth/camera_info
```

```
/camera/depth/points
```

```
/camera/ir/image_raw
```

هذا يعني أننا الآن لدينا

RGB Image

Depth Image

PointCloud

Camera Calibration

TF

في المشروع **Machine Vision** وهذا بالضبط ما نحتاجه لبدء

أين نحن في المشروع الآن

إنتهينا من

Phase 1 — Camera Setup

- تشغيل الكاميرا
- تشغيل depth
- تشغيل point cloud
- calibration
- تلقائي calibration تحميل

الآن نبدأ

Phase 2 — استخراج الإحداثيات ثلاثية الأبعاد

الهدف

تحويل أي نقطة في الصورة

pixel (u,v)

إلى

3D point (X,Y,Z)

وهذا هو أساس

- ضغط زر الأسانسير
- تحديد المقبض
- تحديد باب الأسانسير

أول اختبار سنعمله

سنستخدم **RViz** لنرى العلاقة بين الصورة وال PointCloud.

جديد Terminal افتح

```
ros2 run rviz2 rviz2
```

إعداد RViz

نقم بالآتي RViz في

Fixed Frame

أجعلهُ

```
camera_link
```

أضف Image

أضغط

```
Add → By topic
```

أختر

```
/camera/color/image_raw
```

أضف PointCloud

أضغط

```
Add → By topic
```

أختر

```
/camera/depth/points
```

ماذا يجب أن ترى

أستري

☒ الصورة في نافذة

☒ نقطة ثلاثية الأبعاد في الفضاء

ألو وضعت يدك أمام الكاميرا يجب أن ترى

مسحابة نقاط تمثل يدك في الفضاء

اختبار مهم جدًا

نضع جسمًا صغيرًا مثل

● قلم

● إصبعك

● زر

أمام الكاميرا

ثم راقب

مل النقاط الثلاثية الأبعاد تظهر في نفس مكان الجسم في الصورة؟

إذا كان التطابق جيد فهذا يعني

RGB ↔ Depth alignment صحيح

وهذا مهم جدًا قبل بدء اكتشاف الأزرار

الخطوة القادمة بعد هذا الاختبار

في المشروع تقوم بـ Node سنقوم بإنشاء أول

pixel → 3D point

أي أننا نضغط على أي نقطة في الصورة ونحصل على

X Y Z

وهذا هو الأساس الذي سيجعل الروبوت يعرف مكان زر الأسانسير

سؤال سريع

RViz: بعد فتح

يظهر أمامك أم الشاشة سوداء؟ PointCloud هل

يوفر عليك كل هذا الإعداد (ملف .rviz) جاهز للمشروع كله RViz لو أردت، أستطيع أيضًا أن أعطيك إعداد

مش مفروض دلوقتي اتأكد من العمق واعرف بيقيس عمق وولا ايه واتأكد انه بيتيسه صح ؟

s
a
i
d
:

سؤالك في مكانه تمامًا يا عبدالله

Vision. يعمل ويقيس المسافة بشكل صحيح | قبل أن نكمل أي خطوة **depth** نعم | المفروض الآن نتأكد أن الـ

depth. هذه خطوة أساسية في أي نظام روبوتات يعتمد على

سنقوم بثلاث اختبارات مرتبة مثل ما يعمل مهندس روبوتات في المعمل

اختبار أن العمق يعمل — Phase 2.1

جديد واكتب Terminal افتح

```
ros2 run rviz2 rviz2
```

عداد RViz

Fixed Frame

اجعله

camera_link

depth image أضف

اضغط

Add → By topic

اختر

/camera/depth/image_raw

يجب أن ترى صورة رمادية

عادة

● الأبيض = قريب

● الأسود = بعيد

يعمل depth stream إذا ظهرت الصورة إذن الـ

Phase 2.2 — اختبار القياس الحقيقي للمسافة

الآن نريد التأكد أن الكاميرا تقيس المسافة بشكل صحيح

خذ مسطرة أو متر

ضع جسمًا أمام الكاميرا على مسافة

1 متر تقريبًا

ثم نفذ الأمر

```
ros2 topic echo /camera/depth/image_raw
```

نستري قيم مثل

980

1005

1012

هذه القيم بوحدة

millimeters

أي

$1000 \approx 1 \text{ meter}$

إذا كانت القيم حول 1000 فهذا يعني أن العمق صحيح

Phase 2.3 — اختبار PointCloud

أضف RViz في

```
/camera/depth/points
```

نستري سحابة نقاط ثلاثية الأبعاد

حرك يدك أمام الكاميرا

يجب أن ترى يدك تتحرك في الفضاء ثلاثي الأبعاد

إذا حدث هذا إذن

Depth pipeline ✓

3D reconstruction ✓

اختبار احترافي سريع

نضع جسمًا أمام الكاميرا على

50 cm

depth: القيمة المتوقعة في

500 mm

إذا كانت القيمة قريبة من ذلك فالقياس جيد

أين نحن الآن في المشروع

بعد هذه الاختبارات نكون انتهينا من

Camera setup ✓

Calibration ✓

Depth validation ✓

حقيقية Vision والخطوة القادمة ستكون أول خطوة

Phase 3 — استخراج إحداثيات ثلاثية الأبعاد

تقوم بتحويل Node سنقوم بعمل

pixel (u,v)

إلى

X Y Z

فهذا هو أساس

● ضغط زر الأسانسير

● فتح المقبض

● تحديد باب الأسانسير

سؤال مهم

RViz: عندما أضفت في

/camera/depth/image_raw

هل ظهرت صورة رمادية أم الشاشة سوداء؟

Y
o
u
s
a
i
d
:

حائط الكرتون عند متر والارقام غير منطقيه

ممتاز أنك عملت الاختبار يا عبدالله

والصورة التي أرسلتها تكشف المشكلة مباشرة، وهي ليست في الكاميرا نفسها بل في طريقة قراءة العمق

دعني أشرح لك ماذا يحدث

أولاً : العمق يعمل فعلاً

نفي التيرمينال ظهرت القيم

54

8

54

8

وهذه غير منطقية لمسافة 1 متر لأن المفروض يكون

$\approx 1000 \text{ mm}$

لكن المشكلة هنا أن

ros2 topic echo /camera/depth/image_raw

من الصورة bytes لا يعرض الصورة كعمق حقيقي، بل يطبع

هو: `/camera/depth/image_raw` لأن

`sensor_msgs/Image`

وليس قيمة عمق مباشرة

لماذا الأرقام تبدو خاطئة

رسالة الصورة تحتوي

`uint16 array`

يطبعها كـ `echo` لكن

`bytes`

ولذلك ترى أرقام مثل

54

8

بينما القيمة الحقيقية هي

$54 + (8 \ll 8)$

أي تقريباً

$\approx 2102 \text{ mm}$

وهذا منطقي

الطريقة الصحيحة لاختبار العمق

سنستخدم **RViz + Measure Tool**.

الخطوة 1

غير RViz في

`Fixed Frame`

من

`map`

إلى

camera_link

لأن الخطأ الظاهر عندك

Frame [map] does not exist

الخطوة 2

RViz: في

اضغط

Add

يتم أضاف

/camera/depth/points

(DepthCloud)

الخطوة 3

ثلاثية الأبعاد **Point Cloud** الآن ستظهر

الخطوة 4

في الأعلى اختر

Measure

يتم

● اضغط نقطة على الكاميرا

● اضغط نقطة على الكرتونة

سيظهر لك

distance = ?

لو كانت قريبة من

1 m

إذن العمق صحيح

ملاحظة مهمة من صورتك

أنت أضفت RViz في

DepthCloud

لكن لم يظهر شيء لأن

Fixed Frame = map

وهذا خطأ

تغيره إلى

camera_link

وسيفيظهر كل شيء

لماذا هذا مهم جداً للمشروع

لأن كل المشروع يعتمد على المعادلة

pixel + depth → 3D coordinate

أي

$(u, v, Z) \rightarrow (X, Y, Z)$

وهذا ما يمكن الروبوت من

- معرفة مكان زر الأسانسير
- معرفة مكان المقبض
- تحديد باب الأسانسير

الخطوة التالية بعد التأكد من العمق

سنفعل شيئاً مهماً جداً في المشروع

الحصول على الإحداثيات ثلاثية الأبعاد لأي نقطة نضغط عليها في الصورة

Vision Manipulation. وهذا أول جزء حقيقي من

قبل أن نكمل

وأرسل صورة RViz فعل خطوتين في

☐ غير

Fixed Frame

☐ إلى

camera_link

☐ أضف

/camera/depth/points

ثم أرسل صورة النتيجة

Y
o
u
s
a
i
d
:

مفیش مسافه ظاهره

ممتاز يا عبدالله

بعمل بشكل صحيح PointCloud الصورة التي أرسلتها مهمة جدًا لأنها توضح أن

لكن سبب أنك لا ترى المسافة واضح في الصورة

دعني أوضح لك

أولاً : المسافة ظهرت فعلاً

نظهر RViz في أسفل

Length: 0.445615 m

بحسب المسافة RViz أي أن

القيمة:

0.445 m $\approx$ 44.5 cm

وهذا منطقي إذا كانت يدك أقرب من الكرتونة

لماذا لا تراها بوضوح

يعرض المسافة في الشريط السفلي فقط وليس كرقم كبير في الشاشة RViz

المكان:

أسفل يسار النافذة

Length: 0.445615 m

كيف تقيس المسافة بشكل صحيح

استخدم أداة القياس

في الأعلى اضغط

Measure

تم

اضغط نقطة على الجسم

اضغط نقطة ثانية

وسيطهر

Length: X m

اختبار دقة العمق

دعنا نعمل اختبار حقيقي الآن

خذ متر وضع الكرتونة على

1 meter

تم

Measure ضغط

نقطة على الكاميرا

نقطة على الكرتونة

القيمة المفروض تكون تقريباً

0.9 - 1.1 m

فهذا طبيعي لأن

Depth camera error $\approx$ 2-5%

شيء مهم ألاحظه في صورتك

جيد جداً PointCloud

نرى

● يدك

● الكرتونة

● المكتب

فهذا يعني أن

Depth reconstruction ✓

أي أن الكاميرا تحسب العمق فعلاً

أين نحن الآن في مشروعك

وصلنا إلى هذه المرحلة

Camera ✓

Calibration ✓

Depth ✓

PointCloud ✓

وهذا يعني أن كل نظام الرؤية أصبح جاهزاً

الخطوة القادمة في المشروع (المهمة)

الآن سنبدأ الجزء الحقيقي الذي سيستخدمه الروبوت

تقوم بـ Node سنقوم بعمل

pixel (u,v)

↓

3D coordinate (X,Y,Z)

أي إذا ضغطت على زر في الصورة سيخبرك النظام

X = ?

Y = ?

Z = ?

وهذا هو ما سيجعل الروبوت

● يمد الفراغ

● يضغط زر الأسانسير

سؤال مهم قبل أن نكمل

في الصورة التي أرسلتها

هل الكرتونة فعلاً على مسافة تقريباً نصف متر من الكاميرا؟

لأن القياس الذي ظهر

0.445 m

وهذا قريب جداً من نصف متر

rviz طيب انا معارفشي نقطة الكاميرا فين لانها مش ظاهره في

o

سؤالك ممتاز يا عبدالله

a

t

p

r

d

d

d

d

d

d

d

d

sensor فقط وليس مكان الـ PointCloud يعرض الـ RViz نقطة الكاميرا لا تظهر تلقائياً، لأن RViz فعلاً في

نفسه. لكن يوجد طريقتان صحيحتان لمعرفة مكان الكاميرا

عادة ROS سأعطيك الطريقة التي يستخدمها مهندسو

الطريقة 1 (الأصح) — إظهار

TF. في نظام Frame الكاميرا موجودة كـ

RViz: أضف في

Add → TF

بعد إضافته سترى محاور مثل

camera_link

camera_depth_frame

camera_color_frame

camera_color_optical_frame

:المحاور الثلاثة

● X = الأحمر

● Y = الأخضر

● Z = الأزرق

المكان الذي تظهر فيه هذه المحاور هو مكان الكاميرا الحقيقي

الطريقة 2 — وضع محور للكاميرا

يمكنك أيضاً إضافة

Add → Axes

ثم في الخصائص اجعل

Reference Frame = camera_link

وسيفظهر محور الكاميرا مباشرة

ROS فهم مهم في

أساسي Frame في مشروعك الكاميرا لها

camera_link

وكل شيء يُحسب بالنسبة له

يعني

3D point = camera_link بالنسبة لـ

مثلاً إذا حصلت على

X = 0.2

Y = -0.1

Z = 1.0

فهذا يعني

● 1 متر أمام الكاميرا

● 20 سم يمين

● 10 سم أسفل

لماذا هذا مهم لمشروعك

لأن الفراغ لاحقاً سيأخذ النقطة

camera frame

نحولها إلى

robot base frame

نستخدم

TF transformation

ثم يتحرك إلى زر الأسانسير

اختبار بسيط الآن

RViz: في

Add ضغط

TF اختر

وأرسل صورة بعد ظهور المحاور

شيء مهم جداً سنفعله بعد ذلك

في مشروعك Vision الخطوة القادمة ستكون أهم خطوة في

الضغط على أي نقطة في الصورة والحصول على إحداثياتها ثلاثية الأبعاد مباشرة

هذه هي الخطوة التي سنستخدمها لاحقاً في

● تحديد زر الأسانسير

● تحديد المقبض

● توجيه الفراغ

جاهز للمشروع كله يوفر عليك كل هذه الإعدادات RViz لو أردت، أستطيع أيضاً أن أعطيك ملف

تمام خرينا نخرج احداثيات لكل نقطة

ممتاز يا عبدالله

هذه بالضبط الخطوة التي تجعل نظام الرؤية مفيد فعلياً للروبوت: استخراج إحداثيات ثلاثية الأبعاد لأي نقطة في

الصورة

سنفعلها بطريقة عملية وسهلة أولاً، ثم لاحقاً نربطها بالروبوت

سنمشي في 3 خطوات واضحة

الفكرة الأساسية

لدينا:

pixel (u,v)

+

depth Z

نحولها إلى

3D point (X,Y,Z)

باستخدام معادلات الكاميرا

PointCloud (استخراج النقطة من الـ) الطريقة الأسهل الآن

سنأخذ النقطة مباشرة من

/camera/depth/points

الخطوة 1 — تثبيت المكتبات

جديد: Terminal افتح

```
pip3 install numpy opencv-python
```

تم

```
sudo apt install ros-humble-pcl-conversions
```

بسيطة Node الخطوة 2 — إنشاء

workspace: اذهب إلى

```
cd ~/ros2_ws/src
```

أنشئ ملف

```
gedit point3d_extractor.py
```

ضع الكود التالي

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import PointCloud2
import sensor_msgs_py.point_cloud2 as pc2
class PointExtractor(Node):
    def __init__(self):
        super().__init__('point_extractor')
        self.subscription = self.create_subscription(
            PointCloud2,
            '/camera/depth/points',
            self.listener_callback,
            10)
        def listener_callback(self, msg):
            points = pc2.read_points(msg, field_names=('x','y','z'),
            skip_nans=True)
            for i, p in enumerate(points):
```

```

        if i == 50000: # تأخذ نقطة عشوائية
            x,y,z = p
            print("3D Point:")
            print("X:",x)
            print("Y:",y)
            print("Z:",z)
            break
def main():
    rclpy.init()
    node = PointExtractor()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
if __name__ == '__main__':
    main()

```

الخطوة 3 — تشغيل البرنامج

Terminal جديد: افتح

```
cd ~/ros2_ws/src
```

```
python3 point3d_extractor.py
```

سيبدأ بطباعة نقاط مثل

```
X: 0.12
```

```
Y: -0.08
```

```
Z: 0.98
```

نمناها

النقطة تبعد 0.98 متر أمام الكاميرا

تفسير الإحداثيات

في نظام الكاميرا

X = يمين / يسار

Y = أعلى / أسفل

Z = أمام الكاميرا

مثال

```
X = 0.2
```

Y = -0.1

Z = 1.0

يعني

● 1 متر أمام الكاميرا

● 20 سم يمين

● 10 سم أسفل

ما سنفعله في الخطوة التالية

:الخطوة القادمة ستكون أكثر احترافية

:بدلاً من نقطة عشوائية سنجعل النظام

click on pixel

↓

يعطيك

X Y Z

:أي تضغط على زر الأسانسير في الصورة ويحصل الروبوت على إحداثياته مباشرة

لكن قبل أن نكمل

:أريد أن أعرف شيئاً مهماً

:هل تريد النظام النهائي أن يعمل هكذا

الكاميرا ترى لوحة الأسانسير

↓

نكتشف الأزرار

↓

نأخذ إحداثيات الزر

↓

الروبوت يضغطه

أم أنكم ستختارون الزر يدويًا من الصورة؟


```
ysudo apt install ros-humble-pcl-conversions [sudo] password for abdullah:
o
Reading package lists... Done Building dependency tree... Done Reading state
information... Done The following packages will be upgraded: ros-humble-pcl-
a
conversions 1 upgraded, 0 newly installed, 0 to remove and 380 not upgraded.
d
Need to get 0 B/13.2 kB of archives. After this operation, 0 B of additional disk
space will be used. debconf: unable to initialize frontend: Dialog debconf:
(Dialog frontend requires a screen at least 13 lines tall and 31 columns wide.)
debconf: falling back to frontend: Readline (Reading database ... 346878 files
and dir ectories currently installed.) Preparing to unpack .../ros-humble-pcl-con
versions_2.4.5-2jammy.20260219.022842_amd64.deb ... Unpacking ros-
humble-pcl-conversions (2.4. 5-2jammy.20260219.022842) over (2.4.5-2jam
my.20251228.095228) ... Setting up ros-humble-pcl-conversions (2.4 .5-
2jammy.20260219.022842) ... abdullah@abdullah-sherif:~/ros2_ws$ sudo apt
install ros-humble-pcl-conversions Reading package lists... Done Building
dependency tree... Done Reading state information... Done ros-humble-pcl-
conversions is already the newest version (2.4.5-2jammy.20260219.022842).
ros-humble-pcl-conversions set to manually installed. 0 upgraded, 0 newly
installed, 0 to remove and 380 not upgraded.
abdullah@abdullah-sherif:~/ros2_ws$
```

تمام يا عبدالله

الذي أرسلته طبيعي جدًا ويعني أن كل شيء تم بنجاح log إلى

ماذا حدث

النظام كتب

ros-humble-pcl-conversions is already the newest version

هذا يعني

المكتبة مثبتة بالفعل

لا تحتاج تثبيتها مرة أخرى

PointCloud يمكنك استخدام

والرسالة

debconf: falling back to frontend: Readline

ليست خطأ، فقط لأن التيرمينال صغير

الآن ننتقل للخطوة التالية

سنشغل برنامج استخراج النقاط ثلاثية الأبعاد

إذهب إلى

```
cd ~/ros2_ws/src
```

ثم أنشئ الملف

```
gedit point3d_extractor.py
```

ضع الكود التالي بالكامل

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import PointCloud2
import sensor_msgs_py.point_cloud2 as pc2
class PointExtractor(Node):
    def __init__(self):
        super().__init__('point_extractor')
        self.subscription = self.create_subscription(
            PointCloud2,
            '/camera/depth/points',
            self.listener_callback,
            10)
    def listener_callback(self, msg):
        points = pc2.read_points(msg, field_names=("x","y","z"),
skip_nans=True)
        for i, p in enumerate(points):
            if i == 100000: # نقطة من السحابة
                x,y,z = p
                print("-----")
                print("3D POINT")
                print("X:",x)
                print("Y:",y)
```

```

print("Z:",z)
print("-----")
break
def main():
    rclpy.init()
    node = PointExtractor()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
if __name__ == '__main__':
    main()

```

تشغيل البرنامج

Terminal جديد: افتح

```

cd ~/ros2_ws/src
python3 point3d_extractor.py

```

:المفروض يبدأ يطبع مثل

```

3D POINT
X: 0.21
Y: -0.05
Z: 0.98

```

تفسير الإحداثيات

:في نظام الكاميرا

X = يسار / يمين
Y = أسفل / أعلى
Z = المسافة أمام الكاميرا

مثال

```

X = 0.2
Y = -0.1
Z = 1.0

```

:يعني

● 1 متر أمام الكاميرا

● 20 سم يمين

لماذا هذه الخطوة مهمة لمشروعك

:هذا هو الأساس الذي سنستخدمه لاحقاً في

كشف زر الأسانسير

↓

معرفة مكانه ثلاثي الأبعاد

↓

إرسال الإحداثيات للروبوت

↓

الفرع تضغط الزر

المرحلة التالية (مهمة جداً)

:بعد أن يعمل هذا الكود سنجعله يفعل شيئاً أكثر قوة

:بدلاً من نقطة عشوائية سنجعل النظام

في الصورة pixel اضغط على أي

↓

X Y Z بحسب

manipulation. وهذا بالضبط ما يستخدم في روبوتات

قبل أن نكمل

:عندما تشغل

python3 point3d_extractor.py

أخبرني

:هل ظهرت قيم مثل

X:

Y:

Z:

:أم ظهر خطأ؟

ظهرت

o

ممتاز يا عبدالله

h

a

t

p

r

s

a

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

d

في هذا يعني أن أهم خطوة في نظام الرؤية نجحت **X Y Z** طالما ظهرت قيم

دعني أوضح ماذا يعني ذلك بالنسبة لمشروعك

ماذا حققنا الآن

أصبح عندك النظام التالي يعمل

RGB Camera ✓

Depth Camera ✓

PointCloud ✓

Camera Calibration ✓

3D Point Extraction ✓

وهذا يعني أن الكاميرا تستطيع الآن تحويل أي نقطة في الصورة إلى

X Y Z

في الفراغ الحقيقي

Manipulation. وهذا أساس أي روبوت

مثال عملي مما حدث

هو البرنامج طبع مثلاً

X = 0.12

Y = -0.04

Z = 0.98

هذا يعني

- 98 cm الجسم أمام الكاميرا
- 12 cm يمين
- 4 cm أسفل

أي أن الكاميرا تعرف مكان الجسم بدقة في الفراغ

لماذا هذا مهم جداً لمشروعك

المشروعك يحتاج أن يفعل الروبوت

يرى زر الأسانسير

↓

يعرف مكانه ثلاثي الأبعاد

↓

يوجه الفراع نحوه

↓

يضغط الزر

والآن أصبح الجزء الأصعب من هذه السلسلة يعمل بالفعل

لكن ما فعلناه الآن محدود

PointCloud الكود الحالي يأخذ نقطة عشوائية من الـ

نحن نحتاج أن نفعل هذا بدقة أكبر

:المرحلة القادمة ستكون

محدد Pixel من D استخراج 3

:أي

نضغط على نقطة في الصورة

↓

النظام يعطي

X Y Z

مثلاً نضغط على زر الدور الثالث في الصورة

يحصل النظام على إحداثياته مباشرة

بعدها مباشرة سنبنّي

:الخطوة التالية في المشروع ستكون

Button Detection

نباستخدام

● OpenCV

أو

● YOLO

نتم

centroid button

↓

pixel (u,v)

↓

X Y Z

شيء مهم قبل أن نكمل

أريد أن أعرف

هل تريد نظام المشروع أن يحدد الأزرار

تلقائياً (Vision AI)

مثل

YOLO detects elevator buttons

م

شبه يدوي

يعني

تضغط على الزر في الصورة

والروبوت يضغطه

الفرق بينهما كبير في التعقيد وسنختار المسار الصحيح حسب مشروعك

لا تلقائي طبعاً

o

u

s

a

i

d

.

ممتاز جدًا يا عبدالله

بما أنك اخترت التحديد التلقائي للأزرار فهذا هو المسار الاحترافي لمشروعك، وهو مناسب جدًا

inspection robot arm. لمشروع

دعنا نرتب ما سنفعله كمهندس ميكاترونكس بشكل منطقي

Architecture نظام الرؤية في مشروعك

:النظام النهائي سيعمل بهذا التسلسل

RGB image

↓

Button Detection (YOLO)

↓

pixel (u,v)

↓

Depth

↓

3D coordinate (X,Y,Z)

↓

Transform to robot frame

↓

Robot arm moves

↓

Press button

Phases القادمة لنظام Vision

سنمشي في 4 مراحل واضحة

Phase 3 — Elevator Button Detection

:الهدف

.الكاميرا ترى لوحة الأسانسير وتحدد الأزرار تلقائيًا

:مثلاً النظام يكتشف

Button 1

Button 2

Button 3

نُعطِي

center pixel (u,v)

نُستخدَم

YOLOv8

لأنه

● سريع

● بسهولة ROS يعمل في

● ممتاز للأجسام الصغيرة مثل الأزرار

Phase 4 — 3D Localization

نُبعد اكتشاف الزر

pixel (u,v)

نُستأخذ

depth(u,v)

نُحسب

$X = (u - cx) * Z / fx$

$Y = (v - cy) * Z / fy$

$Z = \text{depth}$

فَيُصبح لدينا

3D position of button

Phase 5 — Transform الروبوت إلى Frame

نُحدث الآن بالنسبة لـ

camera_link

نُمكن الروبوت يحتاجها بالنسبة

robot_base

نُذلك سنستخدم

Phase 6 — Motion Planning

ثم نرسل الإحداثيات إلى

Movelit

ليتحرك الفراغ ويضغط الزر

ما سنبدأ به الآن

سنبدأ بالخطوة الأولى

لاكتشاف أضرار الأسانسير **YOLO** تدريب نموذج

لكن قبل ذلك يجب أن أعرف شيئاً مهماً عن البيئة التي سيعمل فيها الروبوت

سؤالين مهمين جداً

لوحة الأسانسير في مشروعكم

نهل هي

● لوحة حقيقية في مبنى

أم

● لوحة مصنوعة في المعمل؟

الأضرار شكلها

نهل هي

● دائرية

أم

● مربعة

أم

● أضرار لمس؟

لأن هذا سيحدد
نهل سنستخدم

general object detection

أم

custom trained model

خطوة مهمة جدًا
نلو أمكن أرسل صورة لـ

لوحة الأسانسير التي سيعمل عليها الروبوت

الرؤية بالكامل بدقة **pipeline** حتى أستطيع تصميم

حاليا انا مطبع لوحة اسانسير علي ورق وهلزقها علي الحيطه وندرب عليها

ممتاز يا عبدالله

هذا قرار ذكي جدًا هندسيًا لأنك تقلل تعقيد المشروع وتستطيع التحكم في البيئة أثناء التطوير. هذا بالضبط ما يفعله

معظم فرق الروبوتات في البداية

دعنا نصمم النظام بحيث يعمل 100% مع اللوحة المطبوعة ثم يمكن لاحقًا تعميمه على لوحة حقيقية

أولاً : تصميم لوحة الأسانسير للتدريب

ننصحك أن تطبع لوحة بسيطة مثل

[1]

[2]

[3]

[4]

[5]

أو مثل

[1] [2]

[3] [4]

[5] [6]

المهم

- الأزرار واضحة
- الخلفية ثابتة
- حجم الزر حوالي

3 - 5 cm

حتى تستطيع الكاميرا رؤيته جيدًا

خطة تدريب النموذج

سنستخدم

YOLOv8

لأن

- سريع
- Python يعمل بسهولة مع
- مناسب للأجسام الصغيرة

Phase 1 — جمع الصور

نحتاج تقريباً

صورة 50 - 100

اللوحة الأسانسير

التنوع مهم

- مسافات مختلفة

- زوايا مختلفة

- إضاءة مختلفة

مثلاً

0.5 m

1 m

1.5 m

Phase 2 — عمل Annotation

سنستخدم أداة مثل

labelImg

لتحديد مكان كل زر

سنرسم

Bounding box

جول الزر

مثلاً

class = button

أو يمكن

button_1

button_2

button_3

Phase 3 — تدريب YOLO

سنستخدم

ultralytics YOLOv8

لتدريب النموذج

النموذج سيخرج

pixel center (u,v)

لكل زر

Phase 4 — حساب الإحداثيات ثلاثية الأبعاد

نبعد اكتشاف الزر

(u,v)

سنأخذ

depth(u,v)

ثم نحسب

X Y Z

التي حسبناها قبل قليل

Phase 5 — تحريك الفراغ

ثم نرسل الإحداثيات إلى

MoveIt

ليتحرك الفراغ ويضغط الزر

Pipeline الكامل للمشروع

سيصبح

Camera

↓

YOLO detect button

↓

pixel center

↓

Depth

↓

3D coordinate

↓

Transform to robot frame

↓

Move robot arm

↓

Press button

نصيحة مهمة جدًا لمشروعك

اجعل لوحة الأسانسير المطبوعة تحتوي

- أرقام واضحة
- أزرار متباعدة قليلًا

detection. حتى يسهل

الخطوة القادمة التي سنفعلها الآن

على جهازك **YOLO** سنبدأ إعداد بيئة

Terminal: اكتب في

pip install ultralytics

ثم أخبرني

بعدها سأريك خطوة مهمة جدًا

إنا دلوقت طبعنا الصورتين دول محاكات لزراير الاسنسير الدائري والمربع وعاملها ابيض واسود لاني مش مهتم بالالوان

ممتاز جدًا يا عبدالله

اللوحتان اللتان طبعتهما مناسبتان جدًا كبداية للتدريب. الفكرة أن نحول هذا المشهد إلى مسألة كشف كائنات بسيطة:

X, Y, Z . اكتشاف "زر" بغض النظر عن شكله (دائري أو مربع)، ثم استخراج مركزه ومنه نحسب.

سأقترح لك خطة عملية قصيرة حتى تصل لنتيجة بسرعة.

1) Classes تعريف الـ

واحدة فقط Class لتبسيط التدريب في البداية، اجعل كل الأزرار

class: button

لا تفرّق الآن بين 0 و 1 و 2 ... الهدف هو موقع الزر فقط

لكل رقم إذا احتجتم Class أو OCR لاحقًا يمكن إضافة

2) (Dataset) جمع الصور

من زوايا ومسافات (Astra) ضع الورقة على الحائط كما ستفعل في التجربة الحقيقية، ثم التقط صورًا بالكاميرا نفسها

مختلفة

استهدف

● 80-120 صورة تقريبًا

● 0.5 m. 0.8 m. 1 m. 1.2 m. المسافات:

● زوايا ميل خفيفة (يمين/يسار/أعلى/أسفل)

- (robustness) (زيادة) بعض الصور بيدك قريبة أو ظل بسيط

حفظ الصور مثلًا في

dataset/images

3) Annotation عمل

labelImg: استخدم أداة مثل

pip install labelImg

labelImg

- افتح مجلد الصور

- جوله Bounding Box لكل زر ارسم

- اختر Class = button.

- حفظ بصيغة YOLO.

بجانب كل صورة .txt سيتم إنشاء ملفات

4) YOLOv8 إعداد

Ultralytics: بما أنك تثبت

pip install ultralytics

elevator.yaml: أنشئ ملف إعداد للبيانات مثل

path: /home/abdullah/dataset

train: images

val: images

names:

0: button

(في البداية يمكن استخدام نفس الصور للتجربة)

5) التدريب

تُنفذ

yolo detect train model=yolov8n.pt data=elevator.yaml epochs=50 imgsz=640

النتائج سيكون نموذجًا مثل

runs/detect/train/weights/best.pt

6) الاختبار

اختبر النموذج على الكاميرا مباشرة

yolo detect predict model=runs/detect/train/weights/best.pt source=/camera/color/image_raw

أو باستخدام كود Python ROS topic. بسيط يقرأ صورة من

النموذج سيعطيك لكل زر

bounding box

confidence

7) الحصول على مركز الزر

Bounding Box: من الـ

$u=2x_{min}+x_{max}, v=2y_{min}+y_{max}$

وهذا هو الـ pixel center.

8) D تحويله إلى 3

كما فعلنا سابقًا

1. `Z=depth(u,v).` خذ

2. استخدم معاملات الكاميرا:

$$X=fx(u-cx)Z, Y=fy(v-cy)Z$$

الناتج:

Button 3D position: (X, Y, Z)

9) إرسال الإحداثيات للفراغ

`camera_link`. لإحداثيات الآن بالنسبة لـ

MoveIt. للروبوت، ثم نرسل الهدف إلى `base_link` نحولها إلى TF باستخدام

ملاحظات على الصورة التي أرسلتها

- الأبيض والأسود لا مشكلة طالما الحواف واضحة
- `contrast`. للخلفية المعدنية المطبوعة جيدة لأنها تعطي
- وجود نموذجين (دائري ومربع) فكرة ممتازة لزيادة تنوع التدريب

نصيحة صغيرة

حتى تكون الأزرار أوضح للكاميرا (مثلاً A3) إذا أمكن، اطبع الورقة بحجم كبير

إذا أردت، يمكنني أيضاً مساعدتك في الجزء التالي مباشرة

نُخرج مباشرة `/camera/color/image_raw` وموضوع YOLO تجمع بين ROS2 node كتابة

button_pixel → button_3D_position

```
طبيب زاي اخذ صور بالكاميرا واحفظها  
o  
u  
s  
a  
i  
d  
:
```

لأفضل طريقة لجمع الصور هي أن نأخذها ROS2 + Astra camera تمام يا عبدالله، بما أنك تستخدم

topic: مباشرة من الـ

/camera/color/image_raw

a
d

Dataset. وسنكتب سكريبت صغير يحفظ الصور تلقائيًا. هذه الطريقة يستخدمها أغلب الناس لبناء

سأعطيك أسهل طريقة

يحتفظ الصور Python الطريقة 1 (الموصى بها) — سكريبت

إنشاء الملف

أذهب إلى

```
cd ~/ros2_ws/src
```

ثم

```
gedit capture_images.py
```

ضع الكود التالي بالكامل

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2
import os
class ImageSaver(Node):
    def __init__(self):
        super().__init__('image_saver')
        self.bridge = CvBridge()
        self.subscription = self.create_subscription(
            Image,
            '/camera/color/image_raw',
            self.listener_callback,
            10)
        self.counter = 0
        self.save_path = "/home/abdullah/dataset/images"
        os.makedirs(self.save_path, exist_ok=True)
        print("Press SPACE to capture image")
    def listener_callback(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, "bgr8")
        cv2.imshow("Camera", frame)
        key = cv2.waitKey(1)
        if key == 32: # space key
```

```

        filename = f"{self.save_path}/img_{self.counter}.jpg"
        cv2.imwrite(filename, frame)
        print("Saved:", filename)
        self.counter += 1
def main():
    rclpy.init()
    node = ImageSaver()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
if __name__ == '__main__':
    main()

```

تثبيت المكتبات

لو لم تكن مثبتة

```
pip install opencv-python
```

و تثبيت cv_bridge:

```
sudo apt install ros-humble-cv-bridge
```

إنشاء مجلد الصور

```
mkdir -p ~/dataset/images
```

تشغيل الكاميرا

Terminal 1:

```
ros2 launch astra_camera astra_pro.launch.xml
```

تشغيل برنامج التصوير

Terminal 2:

```
cd ~/ros2_ws/src
```

```
python3 capture_images.py
```

ستظهر نافذة الكاميرا

أخذ الصور

كلما ضغطت

SPACE

سيتم حفظ صورة

مثلاً

img_0.jpg

img_1.jpg

img_2.jpg

في

~/dataset/images

كم صورة تحتاج؟

أنصحك

صورة 80 - 120

جمع

☒ مسافات مختلفة

☒ زوايا مختلفة

☒ إضاءة مختلفة

مثال لصور مفيدة

التقط صور عند

0.5 m

0.8 m

1 m

1.2 m

وغير زاوية الكاميرا قليلاً

بعد جمع الصور

:الخطوة التالية مباشرة ستكون

للأزرار **Annotation** عمل

ثم

YOLO تدريب

نصيحة مهمة جدًا

:التقط صور للـ لوحتين معًا

☒ الدائرية

☒ المربعة

.حتى يتعلم النموذج الاثنين

بدون الضغط على) لو أردت، في الرسالة القادمة سأريك طريقة أسرع لجمع 200 صورة تلقائيًا في 20 ثانية فقط

SPACE (كل مرة).


```
pip install opencv-python Defaulting to user installation because normal site-
packages is not writeable Requirement already satisfied: opencv-python in
/home/abdullah/.local/lib/python3.10/site-packages (4.13.0.92) Requirement
already satisfied: numpy>=2 in /home/abdullah/.local/lib/python3.10/site-
packages (from opencv-python) (2.2.6)
```

```
abdullah@abdullah-sherif:~/ros2_ws/src$
```

متثبت فعلاً، ومفيش مشكلة في الرسالة اللي ظهرت **OpenCV** ممتاز ☐ كده

ثبت المكتبة في Python الرسالة دي معناها فقط إن

```
/.local/lib/python3.10/site-packages
```

Ubuntu. وده طبيعي جداً في

جاهز للاستخدام **OpenCV** يعني

الخطوة التالية (ضرورية)

OpenCV. إلى ROS موجود لأنه هو اللي يحول صورة **cv_bridge** نحتاج نتأكد أن

نفذ

```
sudo apt install ros-humble-cv-bridge
```

بعد كده مباشرة

أنشئ مجلد الصور

```
mkdir -p ~/dataset/images
```

ثم شغل الكاميرا

Terminal 1:

```
ros2 launch astra_camera astra_pro.launch.xml
```

تأكد أنها تعمل (مثل ما كانت تعمل معك قبل)

ثم شغل برنامج التصوير

Terminal 2:

```
cd ~/ros2_ws/src
```

```
python3 capture_images.py
```

يُستظهر نافذة الكاميرا

التقاط الصور

بكل مرة تضغط

SPACE

سيتم حفظ صورة

مثلاً

img_0.jpg

img_1.jpg

img_2.jpg

في هذا المسار

/home/abdullah/dataset/images

نصائح أثناء التصوير (مهمة للتدريب)

تصوير الأزرار في

مسافات مختلفة

0.5 m

0.8 m

1 m

1.2 m

زوايا مختلفة

يمين / يسار / أعلى / أسفل

حالات مختلفة

- اللوحة كاملة
- جزء من اللوحة
- إضاءة مختلفة

العدد المناسب

الجمع تقريبًا

صورة 100

بعد جمع الصور

الخطوة التالية ستكون

للأزرار **annotation** عمل

YOLO تدريب

استخراج مركز الزر

depth camera حساب العمق من

إرسال الإحداثيات للروبوت

أردت، أفدر أيضًا أعطيك طريقة احترافية أسرع تجمع

صورة في أقل من دقيقة 300

(Automatic dataset capture) كل مرة SPACE بدون الضغط على

